
Título en castellano
Title in English



Trabajo de Fin de Grado
Curso 2025-2026

Autor

David Xu Hu

Director

Antonio Sarasa Cabezuelo

Celia Martín Cubillo

Grado en Ingeniería de Software
Facultad de Informática
Universidad Complutense de Madrid

Título en castellano
Title in English

Trabajo de Fin de Grado en Ingeniería de Software

Autor
David Xu Hu

Director
Antonio Sarasa Cabezuelo
Celia Martín Cubillo

Convocatoria: junio 2026

Grado en Ingeniería de Software
Facultad de Informática
Universidad Complutense de Madrid

11 de mayo de 2026

A ti, _____

Agradecimientos

A los autores de la plantilla LaTeX de la Facultad de Informática y por transitividad a Marco Antonio y Pedro Pablo, autores de TeXiS.

Resumen

Título en castellano

Un resumen en castellano de media página, incluyendo el título en castellano. A continuación, se escribirá una lista de no más de 10 palabras clave.

Palabras clave

Máximo 10 palabras clave separadas por comas

Abstract

Title in English

An abstract in English, half a page long, including the title in English. Below, a list with no more than 10 keywords.

Keywords

10 keywords max., separated by commas.

Índice

1. Introducción	2
1.1. Motivación	2
1.2. Objetivos	3
1.3. Plan de trabajo	3
1.3.1. Metodología	3
1.3.2. Planificación	4
2. Estado de la cuestión	6
2.1. Sistemas hospitalarios	6
2.2. Herramientas de inteligencia de negocio	7
2.3. Interoperabilidad en el ámbito sanitario	7
2.4. Síntesis de las limitaciones	8
3. Descripción del trabajo	10
3.1. Tecnologías	10
3.1.1. Frontend	10
3.1.2. Backend	13
3.1.3. Base de datos	14
3.1.4. Interoperabilidad	15
3.1.5. Despliegue	16
3.2. Arquitectura	16
3.2.1. Frontend	17
3.2.2. Backend	18
3.3. Diseño del sistema	20
3.3.1. Diseño de datos	20
3.3.2. Diseño de interfaz	22
3.4. Descripción funcional	27
3.4.1. Módulo de Usuario	27
3.4.2. Módulo de Paciente	33
3.4.3. Módulo de Agenda	34
3.4.4. Módulo de Diagnóstico	36
3.4.5. Módulo de Tratamiento	37
3.4.5.1. Lógica de negocio	39
3.4.6. Módulo de Documento Clínico	39
3.4.7. Módulo de Cita	42
3.4.8. Módulo de Flujo de Paciente	43

3.4.9. Módulo de Dashboard	44
3.4.10. Módulo de Auditoría	46
3.5. Detalles generales de implementación	47
3.6. Pruebas y validación	47
3.7. Otros paquetes útiles en Typst	47
4. Conclusiones y Trabajo Futuro	48
4.1. Explicaciones adicionales sobre el uso de esta plantilla	48
4.2. Citar	48
Introduction	50
Conclusions and future work	52
Bibliografía	54
A. Especificación de Requisitos	56
A.A. Módulo de administración	56
A.B. Módulo de usuarios	62
A.C. Módulo de pacientes	67
A.D. Módulo de citas	72
A.E. Módulo de diagnósticos	77
A.F. Módulo de tratamientos	81
A.G. Módulo de documentos clínicos	86
B. Título del apéndice B	88

Índice de figuras

Figura 1	Arquitectura cliente-servidor.	16
Figura 2	Arquitectura del backend con responsabilidades divididas en capas.	18
Figura 3	Arquitectura del backend para la comunicacaión con sistemas externos. .	19
Figura 4	Diagrama entidad-relación con la parte de usuarios, agendas, dashboard y flujo de paciente.	21
Figura 5	Diagrama entidad-relación con la parte de registros médicos y sus relaciones y entidades relacionadas.	21
Figura 6	Colores principales de la paleta de la aplicación.	23
Figura 7	Diagrama que ilustra la diferencia entre formularios verticales y cuadriculares.	24
Figura 8	Formulario de crear cita completo, accesible desde la vista de citas general, con zoom para que aparezcan todos los campos.	25
Figura 9	Formulario de crear cita parcial, accesible desde la vista detallada de paciente.	25
Figura 10	Tabla de citas en la aplicación.	26
Figura 11	Tabla de citas anidada dentro de la vista detallada de usuario.	26
Figura 12	Vista general de los usuarios del sistema.	28
Figura 13	Vista del diálogo al intentar dar de baja a un usuario.	28
Figura 14	Vista detallada de usuario.	29

Índice de tablas

Tabla 1	Comparativa de tecnologías populares para el desarrollo de frontend. . .	11
Tabla 2	Comparativa de tecnologías populares para el desarrollo de backend. . .	13
Tabla 3	Comparativa de tecnologías populares de bases de datos.	15
Tabla 4	Modelo de datos de la entidad Usuario.	29
Tabla 5	Relaciones de la entidad Usuario con el resto de entidades del sistema. .	30
Tabla 6	Modelo de datos de la entidad Rol.	30
Tabla 7	Relaciones de la entidad Rol con el resto de entidades del sistema.	30
Tabla 8	Modelo de datos de la entidad Permiso.	30
Tabla 9	Relaciones de la entidad Permiso con el resto de entidades del sistema. .	31
Tabla 10	Modelo de datos de la entidad Paciente.	34
Tabla 11	Relaciones de la entidad Paciente con el resto de entidades del sistema. .	34
Tabla 12	Modelo de datos de la entidad Agenda.	35
Tabla 13	Relaciones de la entidad Agenda con el resto de entidades del sistema. .	35
Tabla 14	Modelo de datos de la entidad Periodo de Agenda.	35
Tabla 15	Relaciones de la entidad Periodo de Agenda con el resto de entidades del sistema.	35
Tabla 16	Modelo de datos de la entidad Diagnóstico.	36
Tabla 17	Relaciones de la entidad Diagnóstico con el resto de entidades del sistema.	36
Tabla 18	Modelo de datos de la entidad Diagnóstico-Usuario.	37
Tabla 19	Relaciones de la entidad Diagnóstico-Usuario con el resto de entidades del sistema.	37
Tabla 20	Modelo de datos de la entidad Tratamiento.	38
Tabla 21	Relaciones de la entidad Tratamiento con el resto de entidades del sistema.	38
Tabla 22	Modelo de datos de la entidad Tratamiento-Usuario.	39
Tabla 23	Relaciones de la entidad Tratamiento-Usuario con el resto de entidades del sistema.	39
Tabla 24	Modelo de datos de la entidad Documento Clínico.	40
Tabla 25	Relaciones de la entidad Documento Clínico con el resto de entidades del sistema.	40
Tabla 26	Modelo de datos de la entidad Archivo Clínico.	41
Tabla 27	Relaciones de la entidad Archivo Clínico con el resto de entidades del sistema.	41

Tabla 28	Modelo de datos de la entidad Documento Clínico-Usuario.	41
Tabla 29	Relaciones de la entidad Documento Clínico-Usuario con el resto de entidades del sistema.	41
Tabla 30	Modelo de datos de la entidad Cita.	42
Tabla 31	Relaciones de la entidad Cita con el resto de entidades del sistema.	42
Tabla 32	Modelo de datos de la entidad Flujo de Paciente.	43
Tabla 33	Relaciones de la entidad Flujo de Paciente con el resto de entidades del sistema.	43
Tabla 34	Modelo de datos de la entidad Evento de Flujo.	44
Tabla 35	Relaciones de la entidad Evento de Flujo con el resto de entidades del sistema.	44
Tabla 36	Modelo de datos de la entidad Dashboard de Usuario.	45
Tabla 37	Relaciones de la entidad Dashboard de Usuario con el resto de entidades del sistema.	45
Tabla 38	Modelo de datos de la entidad Componente de Dashboard.	45
Tabla 39	Relaciones de la entidad Componente de Dashboard con el resto de entidades del sistema.	45
Tabla 40	Modelo de datos de la entidad Registro de Auditoría.	46
Tabla 41	Relaciones de la entidad Registro de Auditoría con el resto de entidades del sistema.	46
Tabla A.1	Requisito 1.1 - Dar de alta a usuario.	57
Tabla A.2	Requisito 1.2 - Modificar datos de usuario.	58
Tabla A.3	Requisito 1.3 - Dar de baja a usuario.	59
Tabla A.4	Requisito 1.4 - Ver usuario.	60
Tabla A.5	Requisito 1.5 - Listar usuarios.	61
Tabla A.6	Requisito 2.1 - Iniciar sesión.	62
Tabla A.7	Requisito 2.2 - Cerrar sesión.	63
Tabla A.8	Requisito 2.3 - Restablecer contraseña.	64
Tabla A.9	Requisito 2.5 - Modificar datos del perfil.	65
Tabla A.10	Requisito 2.6 - Aplicar opción de accesibilidad.	66
Tabla A.11	Requisito 3.1 - Alta de paciente.	67
Tabla A.12	Requisito 3.2 - Modificar datos de paciente.	68
Tabla A.13	Requisito 3.3 - Dar de baja a paciente.	69
Tabla A.14	Requisito 3.4 - Ver paciente.	70
Tabla A.15	Requisito 3.5 - Listar pacientes.	71
Tabla A.16	Requisito 4.1 - Crear cita.	72
Tabla A.17	Requisito 4.2 - Modificar datos de cita.	73
Tabla A.18	Requisito 4.3 - Finalizar cita.	74
Tabla A.19	Requisito 4.4 - Ver cita.	75
Tabla A.20	Requisito 4.5 - Listar citas.	76
Tabla A.21	Requisito 5.1 - Crear diagnóstico.	77
Tabla A.22	Requisito 5.2 - Modificar datos de diagnóstico.	78
Tabla A.23	Requisito 5.3 - Ver diagnóstico.	79

Tabla A.24 Requisito 5.4 - Listar diagnósticos.	80
Tabla A.25 Requisito 6.1 - Crear tratamiento.	81
Tabla A.26 Requisito 6.2 - Modificar datos de tratamiento.	82
Tabla A.27 Requisito 6.3 - Finalizar tratamiento.	83
Tabla A.28 Requisito 6.4 - Ver tratamiento.	84
Tabla A.29 Requisito 6.5 - Listar tratamientos.	85
Tabla A.30 Requisito 7.1 - Crear documento clínico.	86

Capítulo 1

Introducción

Desde las primeras formas de transmisión oral, pasando por la aparición de la escritura y el dibujo como medio de registro, hasta la digitalización de la información en el último siglo, la gestión del dato ha pasado de manera constante por varios hitos evolutivos. En esta etapa más reciente, el rápido desarrollo tecnológico y digital de las últimas décadas ha supuesto una revolución en la forma en que se generan, almacenan y utilizan los datos. Esta transformación ha supuesto dos grandes cambios en el área de la información: la posibilidad de generar volúmenes masivos de datos, y la capacidad de almacenarlos de manera eficiente y accesible.

Las oportunidades consecuentes han tenido un impacto significativo en todos los ámbitos y sectores, puesto que el dato es un concepto transversal. Esto es especialmente cierto en el sector sanitario, un área de alto riesgo donde la gestión de la información es crítica para apoyar la toma de decisiones y el tratamiento adecuado de pacientes.

Los sistemas digitales de información hospitalaria se han cimentado como herramientas fundamentales para tratar tanto datos clínicos, administrativos como operativos. Asimismo, el desarrollo de estándares de interoperabilidad como HL7 FHIR ha facilitado la comunicación entre distintos sistemas sanitarios, contribuyendo a un ecosistema más integrado.

1.1. Motivación

Hoy en día, existe una gran crisis relacionada a la fatiga del personal sanitario. Las largas jornadas de trabajo y la falta de sueño reducen la capacidad cognitiva y deterioran la salud del personal. Esto a su vez incrementa la posibilidad de cometer errores, poniendo en riesgo la salud del paciente.

Aunque no se trate de un factor contribuyente directo, las complejidades y dificultades de uso de los sistemas con los que trata el personal en su día a día pueden contribuir a aumentar su carga de trabajo y frustración laboral.

En este sentido, mejoras en la usabilidad, accesibilidad, y utilidad de estos sistemas podrían reducir el esfuerzo necesario para su uso, reduciendo una fuente de fatiga para los profesionales. Adicionalmente, permitiría incrementar la eficiencia en el desarrollo de tareas.

Por otra parte, la falta de interoperabilidad constituye otro de los grandes problemas que sufren los sistemas de información digitales. A menudo, en entornos hospitalarios, la información del paciente se encuentra distribuida en varios sistemas que no siempre se pueden comunicar de forma eficiente.

Este trabajo nace a fin de abordar estos problemas mediante el desarrollo de un sistema centralizado de uso diario que pueda comunicarse con otros sistemas externos y que sea fácil y cómodo de usar para el personal hospitalario.

1.2. Objetivos

El objetivo principal de este trabajo es desarrollar una aplicación funcional que sirva para la gestión y análisis de información sanitaria en entornos hospitalarios. El sistema debería, en la medida de lo posible, afrontar o mitigar las dificultades descritas en el apartado anterior.

Para lograr este objetivo, se plantean los siguientes objetivos

- Diseñar e implementar las funcionalidades necesarias para la gestión de usuarios, pacientes y citas en el sistema.
- Implementar mecanismos de interoperabilidad que permitan que el sistema pueda comunicarse con otros sistemas externos.
- Desarrollar un sistema de visualización de información sanitaria mediante dashboards.
- Desarrollar una aplicación que modele los procesos y comportamientos principales de un sistema de gestión hospitalaria, aproximándose al funcionamiento de soluciones reales.
- Diseñar una interfaz intuitiva que favorezca la usabilidad y mejore la eficiencia en la realización de las funciones del sistema.

1.3. Plan de trabajo

En este apartado se describe el plan de trabajo a seguir para la consecución de los objetivos descritos en el apartado anterior. La sección está dividida entre la metodología de desarrollo software escogida, y la planificación concreta planteada.

1.3.1. Metodología

El enfoque empleado para el desarrollo del proyecto fue uno obtenido combinando las ventajas de las metodologías tradicionales y ágiles, de forma que se adaptase a las circunstancias concretas del proyecto.

En una fase inicial, la definición formal de los requisitos de la aplicación fue elaborada en una especificación de requisitos software (SRS). Esta decisión se debió a la necesidad de tener una visión global clara del sistema y a la complejidad del dominio en cuestión. Otra razón por la que se optó por esta opción frente a alternativas como las historias de usuario de las metodologías ágiles fue debido a una mayor eficiencia derivada de experiencias previas redactando documentos SRS.

El resto del desarrollo no obedeció de forma estricta los procesos de ninguna metodología tradicional ni ágil. Las fases de diseño, implementación y pruebas se realizaron de una forma incremental basada en módulos. Estos módulos se crearon agrupando las distintas funcionalidades del sistema según la entidad principal involucrada en ellas. Además, se realizaron revisiones continuas durante todo el proyecto de forma iterativa. Este enfoque se escogió en consideración al uso de tecnologías nuevas y necesidad de adaptación progresiva. Así, se evitaron tanto la rigidez de las metodologías tradicionales, como las prácticas orientadas a equipos de las metodologías ágiles.

Asimismo, se mantuvo una comunicación frecuente con los directores del trabajo para poder recibir feedback constante durante el desarrollo. Esto fue especialmente crucial a causa de las exigencias, requisitos y dificultades que presenta la creación de un sistema funcional en el ámbito sanitario.

1.3.2. Planificación

La planificación concreta del proyecto se formuló en base a la metodología explicada anteriormente. Las fases en concreto son las siguientes:

1. **Fase previa:** definición y planteamiento del proyecto con los directores del trabajo.
2. **Fase de requisitos:** elaboración de la SRS y el modelo del dominio.
3. **Fase de implementación base:** desarrollo de los módulos básicos de la aplicación.
4. **Fase de implementación central:** desarrollo de las funcionalidades operativas.
5. **Fase de ampliación:** desarrollo de mecanismos de interoperabilidad y funcionalidades avanzadas.
6. **Fase de pruebas:** corrección de errores, ajuste de detalles para el correcto funcionamiento de la aplicación.
7. **Fase final:** escritura de la memoria del trabajo asistida por documentación del proceso de desarrollo elaborada durante las fases anteriores.

De forma paralela a los pasos definidos en el plan, se llevaron a cabo tareas de revisión, tanto propias como con los directores, durante toda la duración del trabajo.

Capítulo 2

Estado de la cuestión

Existen multitud de sistemas hospitalarios para la gestión de información sanitaria, así como herramientas para el análisis y explotación de esta información. Además, el ámbito de la interoperabilidad sanitaria ha sido desarrollado de forma amplia y hay una extensa bibliografía al respecto.

A continuación se analizará en detalle el estado actual de estos tres temas, ya que son de gran relevancia en el contexto de este trabajo. Interesa, en especial, examinar sus fortalezas y limitaciones como base para comprender la situación presente y posibles problemas que pueda haber.

2.1. Sistemas hospitalarios

Los **sistemas de información hospitalaria (HIS)** engloban todo sistema de gestión de información utilizada en un contexto hospitalario [1]. Por ejemplo, el sistema que utiliza un farmacéutico para el manejo de medicamentos, o el sistema utilizado por un hospital para tareas operativas como la gestión de citas.

Dentro del HIS global que utiliza un hospital, es importante destacar la existencia del **sistema de registros de salud electrónicos (EHR)**. Según la Organización Internacional de Normalización (ISO) [2], un sistema EHR es aquel que se encarga de almacenar toda la información médica de los pacientes en formato digital.

Los términos EHR y HIS se usan a menudo de forma intercambiable, y los sistemas más usados actúan como un HIS global que contiene múltiples subsistemas, incluyendo el propio sistema EHR. Asimismo, también existen hospitales que optan por utilizar múltiples HIS independientes de distintos proveedores [1,3]. Entre los sistemas más utilizados hoy en día se encuentran Epic EHR (Epic Systems), y Cerner (actualmente Oracle Health) [1,3].

Diversos estudios señalan que los problemas principales que sufren estos sistemas son un mal diseño de interfaz, baja usabilidad, funcionalidades limitadas, y la falta de mecanismos de interoperabilidad [1,4]. Estos mencionan además la dificultad de aprendizaje

de los sistemas y formación de usuarios carente como obstáculos considerables en la adopción de estas herramientas por parte de los profesionales sanitarios.

Otro estudio indica que los médicos invierten una cantidad de tiempo operando estos sistemas superior al tiempo empleado tratando pacientes debido a diseños poco eficientes y complejos [3]. Este estudio señala además que su uso puede incrementar las posibilidades de agotamiento laboral en hasta un 30%.

2.2. Herramientas de inteligencia de negocio

La **inteligencia de negocio (BI)** es el proceso de obtener información para ayudar a la toma de decisiones en una organización [5]. Para ello, las herramientas de BI analizan los datos organizativos y convierten los resultados o conclusiones obtenidos en visualizaciones intuitivas, a menudo contenidas en un dashboard.

En el sector sanitario, los análisis que proporcionan las herramientas de BI pueden ser muy útiles para gestionar los recursos de un hospital y optimizar los procesos del día a día. Por lo general, un hospital tiene dos maneras de acceder a estas capacidades.

La primera es a través de herramientas de BI externas, como pueden ser Power BI y Tableau. Una ventaja clara de esta opción es que permite hacer uso de herramientas consolidadas en el mercado, con muchos recursos, y cuya calidad ha sido demostrada. No obstante, puede haber limitaciones en cuanto a la integración embebida de dichas herramientas en las aplicaciones propias del hospital. Además, es importante considerar los costes de contratar otro servicio adicional de un tercero.

Por otro lado, la segunda opción es utilizar las capacidades existentes de los sistemas ya utilizados por el hospital. Los sistemas HIS incorporan cada vez más elementos de visualización como gráficas y métricas en sus interfaces como apoyo al análisis clínico y toma de decisiones [6]. Estas muestran información relevante para sus usuarios, como puede ser el número de citas pendientes para el día, o una gráfica con el número de nuevos pacientes en los últimos meses. Que esta información se encuentre en el HIS del hospital supone una mayor eficiencia comparado a tener que utilizar una aplicación externa. También se evita la posibilidad de tener que gestionar licencias y suscripciones adicionales. Sin embargo, muchos HIS se limitan a mostrar visualizaciones y dashboards predefinidos, con una capacidad de personalización limitada. En estos casos su utilidad puede verse restringida a proporcionar información que, aunque útil, no permite analizar más allá de la superficie de la situación.

2.3. Interoperabilidad en el ámbito sanitario

En esencia, la interoperabilidad busca resolver el dilema atemporal que suponen las barreras entre idiomas y culturas para la comunicación e intercambio de información entre personas, aplicado a los sistemas informáticos.

Un sistema se considera interoperable cuando puede intercambiar y consumir información de otros sistemas [7]. Para que esto funcione, los sistemas que se comunican deben hacerlo bajo una serie de reglas o estándares predefinidos.

Por otra parte, dentro de la interoperabilidad existen varios tipos. Algunos aplican únicamente a ciertos sectores, mientras que otros son independientes al área donde se aplican [8]. Entre estos últimos, destacan las siguientes cuatro [9]:

- **Interoperabilidad técnica:** se centra en la habilidad básica de intercambiar datos entre sistemas, requiriendo infraestructura y protocolos de comunicación.
- **Interoperabilidad sintáctica:** especifica la estructura y formato comunes que sigue la información intercambiada.
- **Interoperabilidad semántica:** asegura que la interpretación del significado de la información intercambiada es común entre sistemas.
- **Interoperabilidad organizativa:** engloba los procesos y políticas necesarios para permitir el intercambio efectivo de información entre organizaciones.

Entre estos, la interoperabilidad sintáctica y la semántica suelen ser las más relevantes en el ámbito sanitario, puesto que la interoperabilidad técnica es un concepto más elemental, y la organizativa es más abstracta.

Los estándares más utilizados para la interoperabilidad sintáctica sanitaria son *Fast Healthcare Interoperability Resources* (FHIR) y *Observational Medical Outcomes Partnership Common Data Model* (OMOP), mientras que para la interoperabilidad semántica sanitaria destacan *Systematized Nomenclature of Medicine – Clinical Terms* (SNOMED CT) y *Logical Observation Identifiers, Names and Codes* (LOINC) [10].

Aunque existan estándares de interoperabilidad como los descritos, bien documentados y desarrollados, el problema radica en que a menudo no se utilizan. Si bien el sector está migrando de forma progresiva desde sistemas basados en papel hacia la gestión digital a través de los EHR y HIS, persisten limitaciones significativas en la capacidad de los sistemas para comunicarse entre sí. [11]. Esto ha resultado en sistemas fragmentados e incompatibles, dificultando el acceso unificado a la información clínica y limitando el impacto positivo que podrían tener las nuevas tecnologías que dependen del dato digital [9,10].

2.4. Síntesis de las limitaciones

Tras analizar el estado de la cuestión respecto a estos tres temas, las principales limitaciones que existen en los sistemas de información hospitalaria se pueden resumir como las siguientes:

- **Diseño de las aplicaciones:** la mayoría de sistemas de información hospitalaria tienen problemas en su diseño, presentando interfaces complejas, poco eficientes y con baja usabilidad. Esto puede causar frustración y fatiga en los usuarios,

contribuyendo a un peor cuidado del paciente y a la posibilidad de cometer errores durante la operación de los sistemas.

- **Dificultades de aprendizaje de los sistemas:** además de las barreras de usabilidad, la falta de formación adecuada en el uso de las aplicaciones reduce inevitablemente la satisfacción del usuario y el partido que le pueden sacar. Disponer de documentación y recursos claros y accesibles podría ayudar a reducir el impacto de estas complicaciones y acelerar la curva de aprendizaje.
- **Dependencia de herramientas externas para el análisis de datos:** las capacidades de inteligencia de negocio integradas en los propios sistemas HIS utilizados por los hospitales suelen ser limitadas. Para evitar los inconvenientes de tener que depender en aplicaciones de terceros, sería interesante el desarrollo y exploración de un sistema que sea capaz de mantener la adaptabilidad de estas herramientas externas estando integrado en la plataforma de uso diario del hospital.
- **Soporte de interoperabilidad poco extenso:** aún existe una cantidad significativa de sistemas sanitarios que no incluyen las funcionalidades necesarias para comunicarse con otros sistemas. Con el uso cada vez mayor de registros de salud electrónicos e información digital, es crucial que las aplicaciones en este ámbito se desarrollen con la interoperabilidad como requisito indispensable.
- **Fragmentación de la información:** más allá de la fragmentación ocasionada por la falta de interoperabilidad entre aplicaciones, cuantos más sistemas independientes haya en juego, más riesgo hay de fragmentación. Un HIS amplio que abanique múltiples servicios bajo un mismo ecosistema minimiza la necesidad de usar distintas aplicaciones.

La solución descrita en este trabajo ha sido desarrollada con estos puntos en mente. Resolver estas cuestiones considerando las circunstancias limitadas en tiempo y recursos del proyecto es poco realista. No obstante, la aplicación intentará en la medida de lo posible abordar estos problemas y sentar las bases de un sistema de información hospitalaria que sea útil para los profesionales sanitarios.

Capítulo 3

Descripción del trabajo

El sistema desarrollado en este trabajo es un sistema de información hospitalaria llamado HealthAxis. Se trata de una aplicación web que cuenta con los siguientes módulos funcionales:

- Gestión de usuarios y autenticación
- Gestión de pacientes
- Gestión de citas médicas y agendas
- Gestión de diagnósticos y tratamientos
- Gestión de documentos clínicos
- Visualización de historial de paciente basada en grafos
- Sistema de dashboards y analítica
- Capacidades de interoperabilidad mediante FHIR

El resto de esta sección describe las decisiones técnicas tomadas durante la elaboración del proyecto, así como explicaciones detalladas sobre las funcionalidades, comportamiento e implementación de cada módulo.

3.1. Tecnologías

En este apartado se describirán las tecnologías empleadas, junto a su utilidad y motivo de inclusión. Además, las tecnologías principales que actúan como base del sistema recibirán una comparativa adicional con otras alternativas a las escogidas finalmente para justificar su elección.

3.1.1. Frontend

Se valoró el uso de distintas tecnologías para el desarrollo del frontend. Las principales opciones que estuvieron en consideración fueron React, Vue, y Angular.

Aspecto	React	Vue	Angular
Descripción	Librería flexible basada en componentes reutilizables altamente extensible	Framework progresivo y ligero basado en componentes	Framework completo y estructurado para el desarrollo de aplicaciones a gran escala
Ecosistema y recursos	Muy amplio	Amplio	Amplio
Curva de aprendizaje	Media	Baja	Alta
Casos de uso	SPAs e interfaces dinámicas	SPAs y páginas ligeras	Aplicaciones grandes o empresariales
Experiencia previa	Media	Baja	Nula

Tabla 1: Comparativa de tecnologías populares para el desarrollo de frontend.

Tras considerar las ventajas y desventajas de cada una de las opciones, descritas en la tabla 1, **React** fue la elección final. Los factores principales que llevaron a esta decisión fueron la flexibilidad y amplio ecosistema de desarrollo de la librería, además de la experiencia previa en el uso de esta comparado al resto de opciones. Esto último es especialmente relevante debido al tiempo limitado para el desarrollo del proyecto. Por otra parte, la arquitectura basada en componentes reutilizables de React es idónea para un sistema de información hospitalaria, donde es necesario disponer de muchas vistas y elementos similares para distintas entidades.

Adicionalmente, se utilizaron varias librerías complementarias soportadas en el entorno de React. Las más notables son:

- **Material UI**

Material UI (MUI) es librería de código abierto de componentes de React basada en el sistema de diseño *Material Design* de Google. Tiene muchos componentes listos para usarse y permite personalizarlos según las necesidades de uso. Esto la hace idónea para una aplicación donde el diseño de la interfaz es importante, pero resulta poco viable implementar una biblioteca de componentes propia dentro del alcance del proyecto.

- **React Router**

React Router es una de las librerías más utilizadas de código abierto para la gestión del enrutamiento en aplicaciones desarrolladas con React. React Router permite crear distintas páginas y navegar entre ellas, además de diversas utilidades que optimizan la creación y uso de rutas.

- **React Hook Form**

React Hook Form (RHF) es una librería de gestión de formularios. React introduce ciertas complejidades en el manejo de formularios que pueden ser difíciles de navegar manualmente. Esta librería abstrae muchas de estas complejidades y proporciona una sintaxis más sencilla y otras utilidades como validación integrada de campos. Asimismo, está implementada de forma que optimiza el rendimiento minimizando el renderizado innecesario de componentes.

- **Axios**

Axios es una librería de código libre de JavaScript para la realización de solicitudes HTTP desde el navegador web. Axios está basada en promesas, y trae varias ventajas respecto a utilizar la funcionalidad de fetch nativa de JavaScript. Entre estas destacan la gestión automática de errores HTTP y el soporte de interceptores, funciones especializadas que se usan de forma similar a los middleware.

- **FullCalendar**

FullCalendar es una de las librerías de JavaScript más populares para crear calendarios, y está diseñada para integrarse perfectamente con React. Las funcionalidades y abstracciones que provee facilitan mucho la inclusión de vistas de calendario, como puede ser un calendario de citas.

- **React Flow**

React Flow es una librería de código abierto que proporciona acceso a un componente personalizable para la creación de interfaces basadas en nodos. Esto es ideal para la elaboración de visualizaciones creativas, diagramas interactivos y representaciones temporales diseñados como grafos.

- **React Grid Layout**

React Grid Layout es una librería ligera que da acceso a un sistema de layout cuadrícula. La librería permite arrastrar, mover y redimensionar de los componentes colocados en el layout, además de otras utilidades adicionales. Su inclusión facilita mucho la creación de sistemas de dashboard interactivos y personalizables.

- **Recharts**

Recharts es una librería de código abierto de gráficos, basada en la librería D3.js. Su simplicidad y amplio soporte la hacen una librería muy popular que suprime la necesidad de crear y gestionar componentes de gráficos manualmente. Junto a React Grid Layout, Recharts proporciona un buen punto de partida para la elaboración de dashboards dado el alcance de un proyecto como este.

- **Playwright**

- **React Testing Library**

- **Jest**

3.1.2. Backend

En el backend, se consideraron distintos frameworks y entornos de desarrollo. Las tres opciones principales fueron Node.js (con Express), Django, y Spring Boot.

Aspecto	Node.js + Express	Django	Spring Boot
Descripción	Entorno basado en JavaScript ligero y flexible para el desarrollo web	Framework monolítico basado en Python con numerosas funcionalidades integradas	Framework empresarial basado en Java, robusto y orientado a arquitecturas complejas
Ecosistema y recursos	Muy amplio	Amplio	Muy amplio
Curva de aprendizaje	Media	Media	Alta
Casos de uso	Aplicaciones web, sistemas en tiempo real y APIs REST	Aplicaciones web completas y desarrollo rápido de backends	Aplicaciones empresariales complejas y sistemas distribuidos
Experiencia previa	Alta	Baja	Nula

Tabla 2: Comparativa de tecnologías populares para el desarrollo de backend.

Para una aplicación web, la combinación de **Node.js y Express** suele ser la opción más eficiente, como figura en la tabla 2. Así como sucedió durante la elección de tecnología frontend principal, esta vez la experiencia previa también fue un factor significativo en la elección.

Se utilizaron además otras librerías complementarias para el desarrollo del backend. Entre ellas, las más notables son:

- **Express**

Express es el framework de aplicaciones web más popular para Node.js, diseñado para el desarrollo de APIs, servidores y aplicaciones web. Es ligero, extensible, y crucial para el desarrollo de un backend sólido junto a Node.js.

- **Express Session**

Express Session es una librería que se utiliza en entornos de Express para la gestión de sesiones de usuario. Se utiliza para mejorar la experiencia de usuario a través de persistencia de inicio de sesión en la web.

- **Express Validator**

Express Validator es una librería para la validación y saneación de datos en el servidor. Esta funciona como un middleware en entornos de Express, y sus funcionalidades permiten prevenir ciertas vulnerabilidades de seguridad y errores de formateo de datos.

- **Redis**

Redis es una base de datos en memoria de código abierto utilizada para almanecamiento temporal de datos. Al guardar los datos en RAM permite leer y escribir de forma ágil y con latencia reducida. Se utilizó junto con Express Session para la implementación de un sistema de persistencia de sesiones altamente eficiente.

- **Sequelize**

Sequelize es una librería ORM (*Object-Relational Mapping*) para Node.js que facilita la interacción con bases de datos SQL. Sequelize acelera el desarrollo mediante métodos que abstraen consultas de SQL complejas, y con soporte integrado para transacciones y definición de relaciones entre entidades.

- **Bcrypt**

Bcrypt es una librería de hashing criptográfico que permite securizar contraseñas convirtiéndolas en hashes. Es muy importante la inclusión de una librería como Bcrypt para gestionar de forma segura las contraseñas de los usuarios de la aplicación.

- **Jest**

- **Supertest**

3.1.3. Base de datos

Para la base de datos se consideraron PostgreSQL, MongoDB y MySQL. Las tres son tecnologías estándar de base de datos, muy utilizadas en entornos de todo el mundo.

Aspecto	PostgreSQL	MongoDB	MySQL
Descripción	Base de datos relacional con objetos sólida, íntegra y flexible	Base de datos NoSQL orientada a documentos y estructuras flexibles	Base de datos puramente relacional ligera y accesible
Ecosistema y recursos	Muy amplio	Amplio	Muy amplio
Curva de aprendizaje	Media	Baja	Media
Casos de uso	Sistemas con datos complejos, análisis de datos y entornos con muchas operaciones de escritura	Big Data, entornos con información no estructurada o semi-estructurada	Aplicaciones web estándar, comercio electrónico y entornos con muchas operaciones de lectura
Experiencia previa	Media	Media	Baja

Tabla 3: Comparativa de tecnologías populares de bases de datos.

En la tabla 2 se pueden apreciar las diferencias entre las tres opciones. La tecnología escogida finalmente fue **PostgreSQL**, debido a varias razones.

En primer lugar, MongoDB se descartó porque en el ámbito sanitario los datos suelen seguir un esquema relacional. La integridad referencial es crucial a la hora de gestionar relaciones complejas, por lo que se consideró que las alternativas de SQL se ajustarían más a las necesidades del proyecto.

En segundo lugar, MySQL fue descartado por algunas limitaciones que presenta frente a PostgreSQL. Los sistemas de información hospitalaria son utilizados por multitud de profesionales simultáneamente. PostgreSQL es más eficiente al tratar operaciones de lectura concurrentes que MySQL. Asimismo, PostgreSQL tiene otras ventajas, como cumplimiento de las propiedades ACID en todas sus configuraciones, y acceso a la flexibilidad de una base de datos NoSQL con el tipo de dato JSONB. Esto último es relevante ya que aunque la mayoría de datos sigan una estructura relacional, hay casos como el almacenamiento de documentos clínicos dinámicos donde puede resultar muy conveniente y eficiente. Si bien MySQL tiene mayor eficiencia en operaciones de lectura, y estas pertenecen también en gran medida al flujo de uso de un HIS, se ha considerado que en su totalidad, PostgreSQL es una mejor opción dadas las características de la aplicación, su uso en entornos hospitalarios.

3.1.4. Interoperabilidad

El proyecto se centra en la **interoperabilidad sintáctica**, y por extensión, en la **interoperabilidad técnica**. Como se mencionó al describir el estado de la cuestión de la interoperabilidad sanitaria, la interoperabilidad sintáctica y la semántica son las más

cruciales a la hora de diseñar un sistema de información hospitalaria que pueda comunicarse con otros. Sin embargo, la interoperabilidad semántica implica la integración de varios mecanismos complejos de estandarización terminológica. Además, se debe construir por encima de la interoperabilidad sintáctica, ya que no tiene sentido realizar un mapeo de términos sanitarios si no hay estructura común donde intercambiarlos. Es por estas razones que, considerando el alcance del proyecto, se focalizó el esfuerzo en desarrollar una aplicación que fuese interoperable de forma técnica y sintáctica.

En relación a esto, el estándar de interoperabilidad sintáctica escogido fue **FHIR**, de HL7. FHIR está orientado al intercambio en tiempo real de datos clínicos, y es de los estándares más utilizados en sistemas sanitarios. Adicionalmente cuenta con una documentación extensa y detallada, hecho que facilita su implementación correcta en el proyecto.

3.1.5. Despliegue

Para el despliegue de la aplicación en producción se utilizaron **Vercel** para el frontend, y **Render** para el backend y la base de datos..

- **Vercel**

Vercel es una plataforma en la nube que permite el despliegue y hosting de aplicaciones web. Está especialmente optimizada para trabajar con tecnologías como React, y ofrece escalado automático y un alto rendimiento de carga de recursos que mejoran la experiencia de usuario. Vercel cuenta además con una infraestructura de seguridad robusta, con protección ante ataques de DDoS y un firewall multicapa.

- **Render**

Render es una plataforma en la nube con soporte para el despliegue de una amplia gama de servicios. Evita que haya momentos donde la aplicación esté caída durante actualizaciones y red despliegues mediante mecanismos de despliegue continuo y sustitución de instancias, maximizando la disponibilidad del sistema. Esto, junto al soporte de escalabilidad automática y diversas funcionalidades de seguridad integradas, la convierten en una opción adecuada como servicio de hosting y despliegue para el backend y base de datos de la aplicación.

3.2. Arquitectura

En términos generales, el sistema sigue una arquitectura cliente-servidor clásica, donde el frontend actúa como cliente y realiza solicitudes al backend. Esta se puede apreciar a continuación en la figura 1.

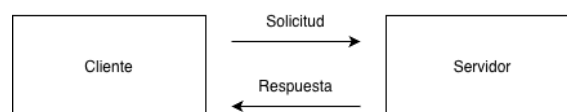


Figura 1: Arquitectura cliente-servidor.

A partir de esta separación general, se distinguen por un lado la organización interna del backend, y por otro la del frontend.

3.2.1. Frontend

El frontend fue desarrollado con una **arquitectura basada en features**, basada en propuestas como la metodología *Feature-Sliced Design* (FSD) y la estructura definida en *Bulletproof React* [12,13]. Una feature consiste en un conjunto de características o funcionalidades de la aplicación autocontenidas. Ejemplos de una feature podrían ser la autenticación del sistema, o la gestión de pacientes.

Entre los rasgos que definen la arquitectura, cabe destacar el principio de evitar realizar importaciones de código entre features. Esto permite que cada feature sea independiente y fácil de modificar sin afectar al resto de features, haciendo que la aplicación sea más escalable. Además, la estructura del proyecto en sí se hace más legible y ordenada, facilitando la colaboración entre desarrolladores.

La estructura organizativa concreta utilizada en el proyecto se define a continuación en la listado 1.

```
src
|
+-- app                // Capa de aplicación del frontend
| |
| +-- pages            // Páginas de la aplicación
| +-- App.jsx          // Componente principal de la aplicación
| +-- router.jsx       // Configuración global de las rutas
| +-- ...
|
+-- features           // Módulos divididos por feature
| |
| +-- patients
| +-- appointments
| +-- ...
|
+-- assets             // Recursos estáticos
| |
| +-- css              // Archivos CSS
| +-- js               // Archivos JavaScript
| +-- img              // Imágenes
|
+-- components         // Componentes reutilizables
+-- shared             // Elementos compartidos entre features
+-- hooks              // Hooks reutilizables
+-- utils              // Funciones auxiliares reutilizables
+-- lib                // Configuración de librerías externas
+-- config             // Elementos de configuración global
+-- main.jsx           // Punto de entrada de la aplicación
```

Listado 1: Estructura de carpetas del frontend.

La carpeta `shared` es necesaria para elementos compartidos, por ejemplo constantes, que normalmente serían propias de cada feature pero que necesitan ser utilizadas en varias features. En el caso de las constantes, aparecen debido a la existencia de formularios con información sobre varias entidades (e.g. selección de usuario en el formulario de crear cita). Aunque se hayan movido estos elementos a una carpeta compartida, podría seguir considerándose como romper el principio de no realizar importaciones entre features. No obstante, se considera que en el caso de este proyecto no debería suponer un problema significativo, puesto que el uso de esta carpeta se ha mantenido al mínimo.

Por otra parte, la estructura organizativa dentro de cada feature es como se muestra en la listado 2.

```
features
|
+-- patients
|   |
|   +-- api          // Llamadas reutilizables al backend
|   +-- components  // Componentes propios de la feature
|   +-- hooks       // Hooks propios de la feature
|   +-- utils       // Funciones auxiliares de la feature
|   +-- config      // Elementos de configuración de la feature
```

Listado 2: Estructura concreta de cada feature.

3.2.2. Backend

El backend fue desarrollado con una **arquitectura basada en capas**, para que la aplicación fuese mantenible y escalable. La división de responsabilidades entre las distintas capas hace el código más legible y permite hacer cambios en cada capa sin romper el resto. En la figura 2 se muestra distribución y flujo concreto de las capas del sistema.

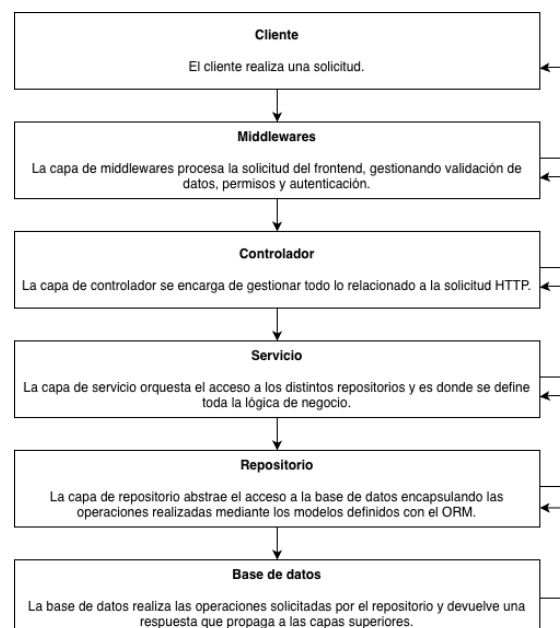


Figura 2: Arquitectura del backend con responsabilidades divididas en capas.

La capa de servicio es especialmente importante porque, como indica la figura 2, es donde se centraliza la lógica de negocio necesaria para que el sistema sea representativo de los flujos de trabajo y reglas de un entorno hospitalario real.

Por otra parte, la comunicación interoperable con sistemas externos a través de FHIR se ha gestionado de forma similar al resto de solicitudes. La única diferencia es la presencia de funciones de mapeo en los controladores destinados a la interoperabilidad que traducen la información interna del sistema al formato interoperable descrito en FHIR. Para mayor claridad, el flujo del proceso y las capas involucradas se han vuelto a mostrar en la figura 3.

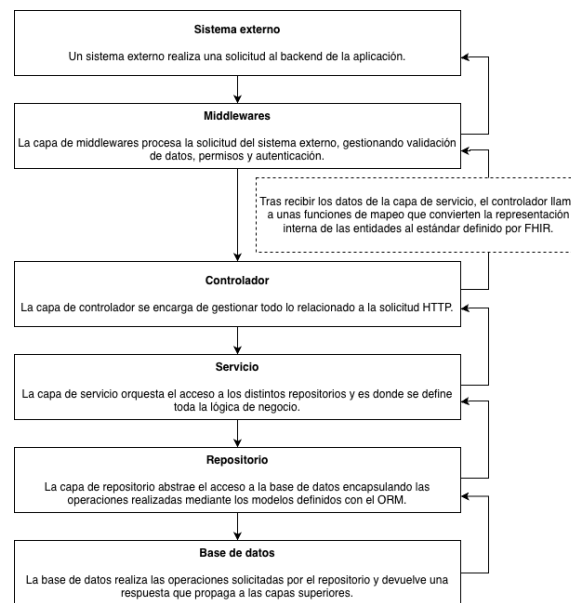


Figura 3: Arquitectura del backend para la comunicación con sistemas externos.

Uniendo los requisitos de la arquitectura por capas, tanto para solicitudes de clientes comunes como para la interoperabilidad con FHIR, y las características generales de un backend organizado, la estructura de carpetas resultante para el backend del proyecto es como se indica en el listado 3.

```

src
|
+-- api                // Acceso HTTP al backend
|   |
|   +-- controllers    // Controladores para las solicitudes HTTP
|   +-- routes         // Definición de rutas de la API
|   +-- validators.jsx // Validación y saneación de datos de entrada
|   +-- fhir           // Capacidades para la conversión a FHIR
|   |   |
|   |   +-- mappers    // Conversión de modelos internos a FHIR
|   |   +-- utils      // Funciones auxiliares para facilitar el mapeo
|   |
+-- config             // Configuración global del backend
+-- middlewares        // Middlewares compartidos del sistema
+-- models             // Definición de entidades y relaciones
+-- repositories       // Repositorios para cada entidad
+-- services           // Servicios para cada entidad
+-- tests              // Pruebas unitarias y de integración
+-- uploads            // Almacenamiento de archivos subidos
+-- utils              // Funciones auxiliares reutilizables
+-- app.jsx            // Construcción del backend
+-- index.jsx          // Punto de entrada e inicialización

```

Listado 3: Estructura de carpetas del backend.

3.3. Diseño del sistema

El sistema ha sido diseñado a través de numerosas decisiones tomadas explícitamente teniendo en cuenta el ámbito sanitario en el que opera la aplicación, el alcance del proyecto, y la experiencia de usuario.

Cabe destacar que debido al carácter iterativo e incremental del proyecto y la gran cantidad de flujos funcionales, se decidió que la especificación de requisitos sería suficiente para guiar el desarrollo por encima de la elaboración exhaustiva de diagramas de comportamiento como los de secuencia.

3.3.1. Diseño de datos

Esta sección describe las decisiones generales del diseño de la base de datos de la aplicación, y algunas decisiones globales importantes que se tomaron al respecto. A fin de no sobrecargar el apartado, las explicaciones concretas del modelo de cada entidad y sus relaciones aparecerán más tarde en el apartado de descripción funcional.

Partiendo con PostgreSQL como tecnología de elección, el modelo de datos se diseñó de forma relacional para poder representar de forma adecuada las distintas entidades de la aplicación y las relaciones sanitarias entre ellas. Con este propósito, se elaboró un modelo entidad-relación (ER) simplificado para representar esta información. El modelo ha sido dividido entre la figura 4 y la figura 5 para facilitar su visualización.

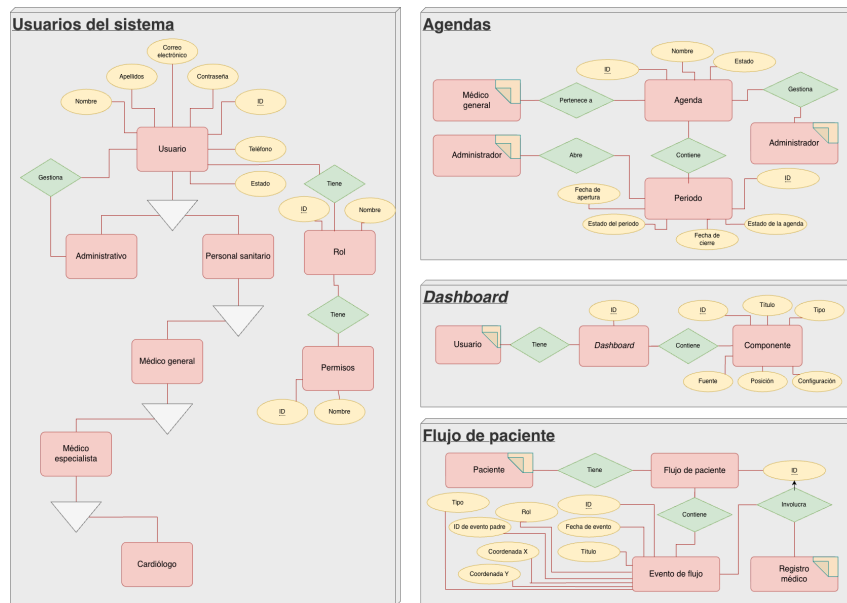


Figura 4: Diagrama entidad-relación con la parte de usuarios, agendas, dashboard y flujo de paciente.

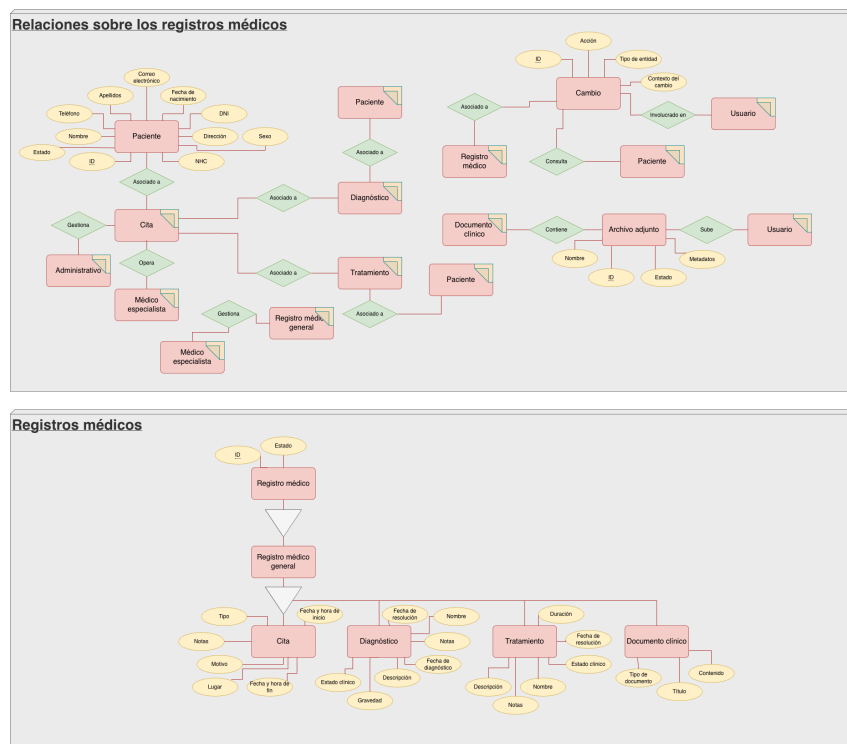


Figura 5: Diagrama entidad-relación con la parte de registros médicos y sus relaciones y entidades relacionadas.

Debido a la gran cantidad de entidades y relaciones, las entidades han sido definidas una vez con todos sus atributos, mientras que las relaciones en las que participan se han distribuido en distintos contenedores agrupados por relación lógica. Las figuras de entidad en dichas relaciones que tienen un icono en su esquina superior derecha

representan entidades que ya han sido definidas en otro punto del diagrama, siendo estas copias utilizadas para mantener el diagrama legible.

Por otra parte, los atributos derivados de relaciones están implícitos en el diagrama. Otros atributos como **UUID**, **createdAt** y **updatedAt** se omitieron por simplicidad. Su relevancia se explicará como parte de las decisiones de diseño descritas a continuación.

- **Uso de generalizaciones:** varias de las generalizaciones se plantearon de cara a futuros desarrollos de la aplicación. Por ejemplo, las relaciones padre-hijo en la sección de usuarios del sistema son redundantes en el sistema actual donde solo existen cardiólogos y administrativos, pero serían necesarias en una versión futura donde hubiese más tipos de especialistas.
- **Identificadores y UUID:** en varias vistas del frontend (e.g. en la vista detallada de paciente), resulta necesario identificar entidades concretas en la URL. No obstante, mostrar identificadores secuenciales en el frontend puede suponer un riesgo de seguridad, al revelar información sobre el volumen y estructura de los datos almacenados en el sistema. Para evitar esto, las entidades que lo necesitan cuentan con un atributo adicional *universally unique identifier* (UUID) como identificador público, manteniendo el ID interno como clave primaria.
- **Información temporal:** todas las entidades incorporan por defecto las columnas **createdAt** y **updatedAt** al usar Sequelize como ORM. Esto permite mantener información temporal sobre las entidades, relevante para tareas de ordenación cronológica y seguimiento histórico de datos.
- **Integridad, trazabilidad y auditoría:** el sistema no permite eliminar datos de forma permanente, únicamente marcarlos como obsoletos, erróneos, o inactivos. En un sistema de información hospitalaria es importante para no perder información importante, mantener integridad referencial de los datos, y motivos de auditoría y trazabilidad.

3.3.2. Diseño de interfaz

La interfaz de la aplicación se planteó con cuidado para tener un aspecto moderno sin estar sobrediseñada, priorizando por encima de todo la usabilidad del sistema.

Durante las primeras fases de diseño se prototiparon varios mockups en Figma para explorar distintas posibilidades de distribución, navegación y aspecto para la interfaz. Estos pueden consultarse en el Anexo **TODO**.

El resto de la sección detalla las decisiones de diseño más significativas tomadas durante el desarrollo del proyecto. Dichas decisiones fueron apoyadas por información de fuentes reputables en la industria del diseño como *Nielsen Norman Group* (NNG).

- **Paleta de color**

La elección de una paleta de color para aplicación es especialmente importante ya que está directamente relacionada con la usabilidad del sistema y la fatiga visual.

El azul suele ser un color que denota tranquilidad y estabilidad [14]. A la hora de operar el HIS y tratar pacientes, es deseable que el profesional sanitario esté calmado, especialmente ante situaciones que puedan causar frustración o ansiedad. Es por eso que el color primario de la aplicación es un azul intermedio, ni muy oscuro ni muy claro. El resto de colores fueron escogidos como paleta complementaria a este color. Parte de la paleta completa se puede ver en la figura 6.

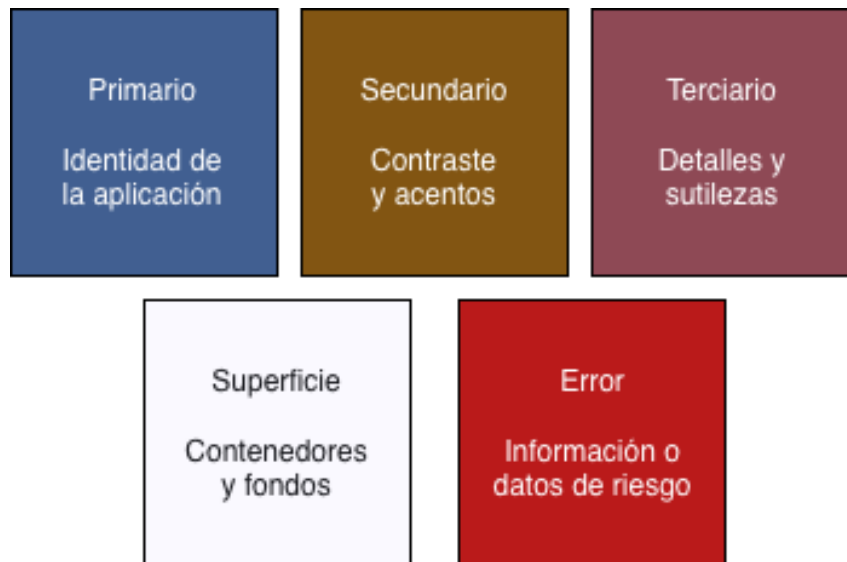


Figura 6: Colores principales de la paleta de la aplicación.

- **Formularios**

El diseño de formularios es crucial a la hora de diseñar un sistema de información hospitalaria. Son elementos donde el usuario va a invertir gran parte de su tiempo, y donde cometer errores podría tener consecuencias significativas.

Existen diversas pautas a seguir para crear formularios claros y estructurados, como agrupar campos similares y establecer un orden lógico de compleción [15,16]. Entre estas directrices destaca de que los formularios con múltiples columnas, aunque ahorran espacio, suelen ser menos intuitivos y llevar a cometer más errores.

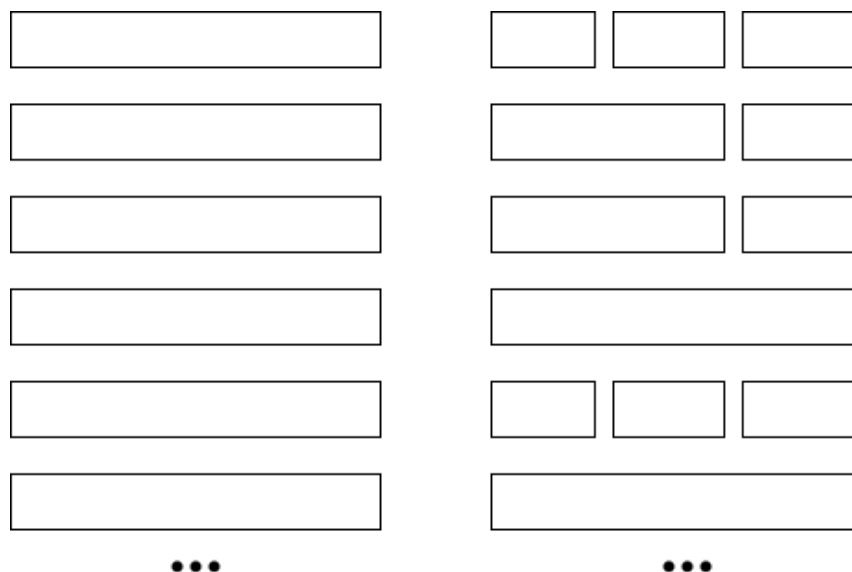


Figura 7: Diagrama que ilustra la diferencia entre formularios verticales y cuadriculares.

El diagrama mostrado en la figura 7 no es del todo representativo, ya que los dos formularios tienen un número de campos distinto. Sin embargo, se ha considerado que es válido a fin de mostrar la diferencia en peso visual entre ambos. Los campos restantes en el formulario vertical o bien aparecerían desplazándose hacia abajo, o bien formarían parte de un formulario multietapa. Ambas alternativas presentan ciertas limitaciones, como la posibilidad de perder contexto al perder de vista algunos campos a medida que se avanza en el formulario. No obstante, se ha considerado que el impacto positivo de utilizar la estructura de columna única en la medida de lo posible compensa las desventajas que trae.

Siguiendo con otro aspecto importante, la gestión de errores en los formularios se diseñó para facilitar que el usuario pueda entender los errores y corregirlos sin sentirse abrumado [17].

Por otro lado, se ha reducido la cantidad de campos a rellenar de forma inmediata en ciertos formularios accesibles desde vistas detalladas de entidad, como puede ser la vista detallada de cita. Los campos removidos de dichos formularios son los que se pueden inferir del contexto de la vista (e.g. el paciente involucrado al añadir un diagnóstico en una cita), y algunos campos opcionales. Estos campos no imprescindibles se pueden rellenar más tarde editando los datos de la entidad. En la figura 8 y la figura 9 se ilustra un ejemplo de esta decisión.

Figura 8: Formulario de crear cita completo, accesible desde la vista de citas general, con zoom para que aparezcan todos los campos.

Figura 9: Formulario de crear cita parcial, accesible desde la vista detallada de paciente.

En este caso concreto, el formulario parcial hace omisión del campo de paciente, puesto que tiene la información de la vista del apciente, y de las notas opcionales. El lugar de la cita se ha conservado al considerarse un campo más importante aunque no sea obligatorio de rellenar. El objetivo de estos cambios es intentar reducir el tiempo invertido en rellenar formularios, especialmente durante procesos como la atención a un paciente durante una cita.

- **Diálogos y páginas completas**

Si bien no hay estudios formales al respecto, existe cierto consenso en la comunidad de UX sobre la diferencia en casos de uso de páginas enteras y diálogos. Con los diálogos el usuario retiene un mayor contexto sobre la acción o contenido que está viendo. No obstante, también existe una mayor carga cognitiva, ya que hay información adicional presente en la pantalla, y el espacio dedicado al diálogo es inevitablemente menor al de una nueva página. Esta carga superior podría llevar a mayores tasas de error, especialmente tras jornadas largas de trabajo.

Para el proyecto, se optó por emplear un patrón de restringir el uso de diálogos a formularios con menos de 6 campos y otros contenidos como alertas y confirmaciones de acción. En cambio, se le destinaron páginas completas a formularios más complejos y vistas con información considerable .

• Tablas

La mayoría de la información del sistema está presentada en formato tabular. Todas las tablas están diseñadas de forma consistente para garantizar una experiencia de usuario cohesiva y predecible. En la figura 10 se aprecia como ejemplo la tabla de citas.

TABLA

CALENDARIO

Busca por motivo, paciente, usuario, estado, fecha

ANADIR CITA

Motivo	Fecha y hora de inicio	Paciente	Profesional	Lugar	Estado	
Revisión	30/04/2026, 10:38	Debbie Jacobson	Adah Fadel	Port Isabella	Registrada	
Revisión	30/04/2026, 10:38	Rose Mayer	Kato Osinski	West Glendaview	Registrada	
Chequeo rutinario	30/04/2026, 10:38	Damon Kub	Tommy Bruen	Catalina Foothills	Registrada	
Resultados de laboratorio	30/04/2026, 10:38	Glenda Oberbru...	Stella Bradtke	Jesustown	Registrada	
Resultados de laboratorio	30/04/2026, 10:38	Hollis Friesen	Stephen Kuvalis	West Jesseside	Registrada	
Chequeo rutinario	30/04/2026, 10:38	Sofia Bradtke	Emil Weber	Krajcikbury	Registrada	
Resultados de laboratorio	30/04/2026, 10:38	Robyn Kirlin	Mauricio Hauk	Port Leahview	Registrada	
Revisión	30/04/2026, 10:38	Maxine Senger	Manuel Larson...	Millford	Registrada	
Consulta especializada	30/04/2026, 10:38	Eleazar Fritsch	Gloria Buckridge	Shanborough	Registrada	

Filas por página: 100 1-100 de 209

Figura 10: Tabla de citas en la aplicación.

Dentro de las vistas detalladas de entidades también aparece información contextual que requiere el uso de tablas. Su diseño se alteró ligeramente para diferenciarlas de las tablas generales sin parecer incoherente. En la figura 11 se muestra la tabla de citas de un usuario, donde se pueden observar las variaciones respecto a la tabla en la figura 10.

CITAS ASOCIADAS

Busca por motivo, paciente, estado, fecha

ANADIR CITA

Motivo	Fecha y hora de inicio	Paciente	Lugar	Estado	
Consulta especializada	03/05/2026, 10:38	Jayson Johnston	East Ana	Cancelada	
Consulta especializada	07/05/2026, 10:38	Diana Bradtke	Kettering	Cancelada	
Chequeo rutinario	11/05/2026, 10:38	Jana Heidenreich	Jakubowskiworth	Programada	
Revisión	02/05/2026, 10:38	Kiera Bogan	Kayborough	Programada	
Chequeo rutinario	30/04/2026, 10:38	Jayson Johnston	Albuquerque	Registrada	
Chequeo rutinario	02/03/2026, 11:38	Diana Bradtke	New Juanita	Completado	

Filas por página: 100 1-8 de 8

Figura 11: Tabla de citas anidada dentro de la vista detallada de usuario.

En ambos casos, las acciones realizables sobre cada elemento de la tabla están recogidas en la columna a la derecha del todo. Aquellas acciones que son menos frecuentes, como editar los datos de la fila, están agrupadas en un menú desplegable para disminuir el

ruido visual. Por otra parte, acciones más prioritarias como realizar el check-in de una cita se muestran fuera de dicho menú para facilitar su acceso.

- **Navegación**

La navegación principal de la aplicación es a través de un *drawer* (cajón de navegación) vertical a la izquierda de la pantalla. Esto tiene varias ventajas, como un menor coste de interacción y mayor descubribilidad [18].

En las vistas anidadas (e.g. la vista detallada de cita) se muestran *breadcrumbs* (miga de pan) con la localización jerárquica del usuario. Esto permite al usuario orientarse y navegar a través de la aplicación sin perder contexto.

3.4. Descripción funcional

Como se adelantó previamente, las funcionalidades del sistema están divididas por módulos lógicos. A lo largo de esta sección, se mostrarán fragmentos de código junto a las descripciones textuales, tablas y figuras. Estos fragmentos han sido simplificados por legibilidad y para adaptarlos al formato de la memoria, omitiendo detalles no relevantes para el contexto de las explicaciones donde aparezcan. No obstante, reflejan fielmente el funcionamiento del código utilizado en el proyecto.

3.4.1. Módulo de Usuario

Todo sistema destinado al uso por parte de personas requiere de un módulo de gestión de usuarios. En el proyecto, permite personalizar la interfaz y experiencia de uso de cada profesional sanitario. Además, es necesario para securizar la aplicación y permitir su uso solo para aquellas personas validadas por el hospital.

Funcionalidades

- Inicio de sesión y autenticación
- Creación y edición de usuarios
- Vista general de usuarios (figura 12)

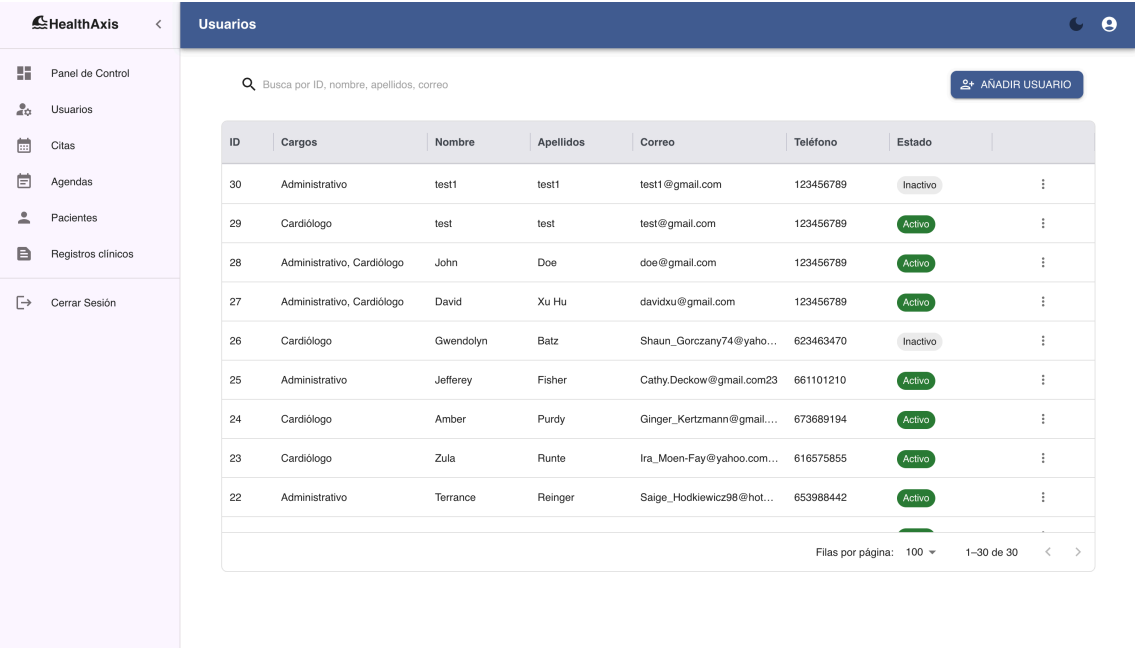


Figura 12: Vista general de los usuarios del sistema.

- Baja lógica y reactivación de usuarios (figura 13)

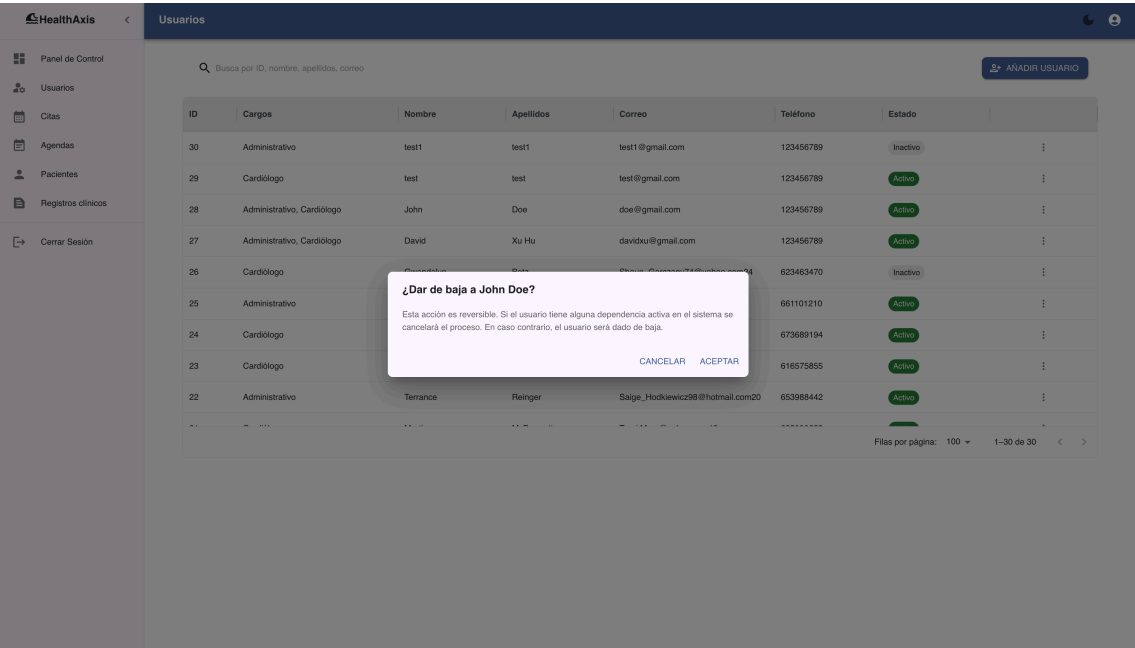


Figura 13: Vista del diálogo al intentar dar de baja a un usuario.

- Vista de perfil y modificación de contraseña
- Vista detallada de usuario (figura 14)

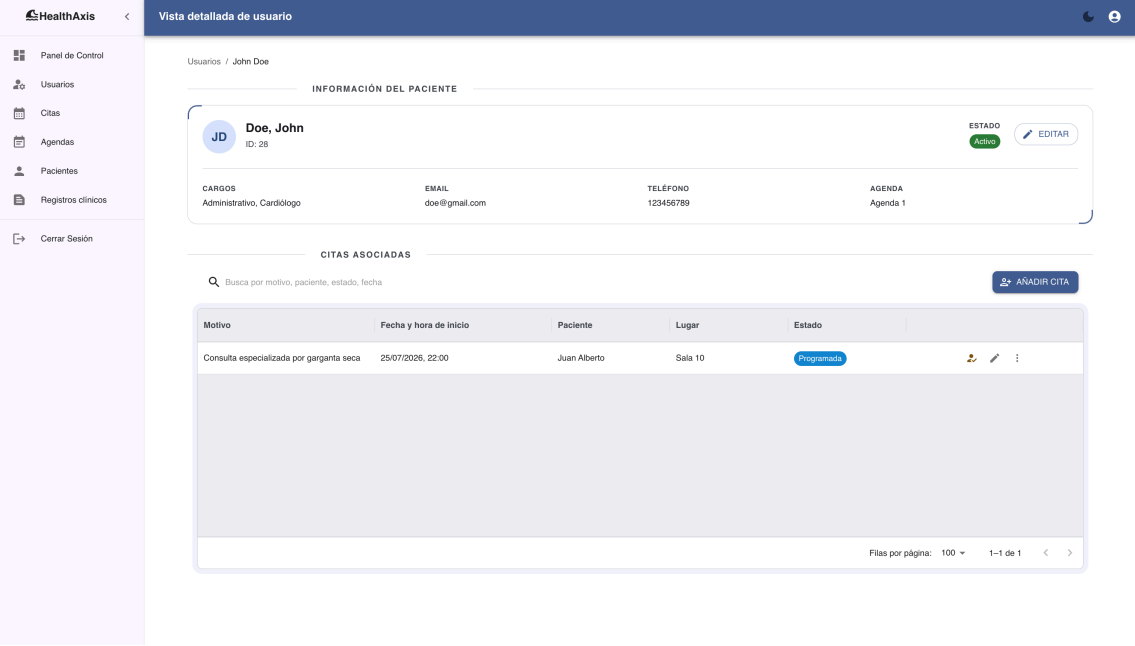


Figura 14: Vista detallada de usuario.

Modelo de datos

Los atributos que tiene la entidad de usuario, así como las relaciones concretas que tiene con el resto de entidades se muestran en la tabla 4 y la tabla 5.

Campo	Tipo	Descripción
id	INTEGER	Identificador interno del usuario
uuid	UUID	Identificador público único del usuario
agendaId	INTEGER	Referencia a la agenda asociada
name	STRING(50)	Nombre del usuario
surname	STRING(60)	Apellidos del usuario
email	STRING(100)	Correo electrónico del usuario
password	STRING(255)	Contraseña cifrada del usuario
phone	STRING(20)	Número de teléfono del usuario
status	ENUM("ACTIVE", "INACTIVE")	Estado del usuario dentro del sistema
createdAt	DATE	Fecha de creación del registro
updatedAt	DATE	Fecha de última actualización del registro

Tabla 4: Modelo de datos de la entidad Usuario.

Entidad relacionada	Cardinalidad
Agenda	$N \rightarrow 1$
Cita	$1 \rightarrow N$
Rol	$N \rightarrow N$
Diagnóstico	$N \rightarrow N$
Tratamiento	$1 \rightarrow N$
Documento clínico	$1 \rightarrow N$
Archivo clínico	$1 \rightarrow N$
Dashboard	$1 \rightarrow 1$
Registro de auditoría	$1 \rightarrow N$

Tabla 5: Relaciones de la entidad Usuario con el resto de entidades del sistema.

En un entorno hospitalario real, un profesional sanitario debería de poder estar en varias agendas, pero este comportamiento se simplificó a una sola agenda por usuario en consideración al alcance del proyecto.

La información sobre el modelo y relaciones de la entidad Rol (tabla 6, tabla 7) y de la entidad Permiso (tabla 8, tabla 9) aparecen también en este apartado debido a su cercanía con la entidad Usuario.

Campo	Tipo	Descripción
id	INTEGER	Identificador interno del rol
name	STRING(50)	Nombre único del rol dentro del sistema
createdAt	DATE	Fecha de creación del registro
updatedAt	DATE	Fecha de última actualización del registro

Tabla 6: Modelo de datos de la entidad Rol.

Entidad relacionada	Cardinalidad
Usuario	$N \rightarrow N$
Permiso	$N \rightarrow N$

Tabla 7: Relaciones de la entidad Rol con el resto de entidades del sistema.

Campo	Tipo	Descripción
id	INTEGER	Identificador interno del permiso
name	STRING(50)	Nombre único del permiso dentro del sistema
createdAt	DATE	Fecha de creación del registro
updatedAt	DATE	Fecha de última actualización del registro

Tabla 8: Modelo de datos de la entidad Permiso.

Entidad relacionada	Cardinalidad
Rol	$N \rightarrow N$

Tabla 9: Relaciones de la entidad Permiso con el resto de entidades del sistema.

Lógica y reglas de negocio

- No pueden existir dos usuarios con el mismo correo (listado 4).

```

async function createUser(userData, roles = []) {
  return await sequelize.transaction(async (t) => {
    const userExists = await UserRepository.findByEmail(...);

    if (userExists)
      throw new ConflictError(
        '...',
        { email: userData.email }
      );
  });
}

```

Listado 4: Comprobación de si el correo ya existe en el servicio de creación de usuario.

- No se puede dar de baja a un usuario con citas activas (listado 5).

```

async function deactivateUser(uuid) {
  return await sequelize.transaction(async (t) => {
    ...
    const activeAppointments =
      await
      AppointmentRepository.hasActiveAppointmentsByUserId(...);

    if (activeAppointments) {
      throw new ValidationError('...', 400, {
        activeAppointments,
      });
    }
    ...
  });
}

```

Listado 5: Comprobación de si el usuario tiene citas activas en el servicio de baja de usuario.

- Los usuarios solo pueden acceder a funcionalidades permitidas por su rol (listado 6).

```
function requirePermission(permission) {  
  return (req, res, next) => {  
    if (!req.user)  
      return next(new ForbiddenError('...'));  
  
    if (!req.user.permissions.includes(permission))  
      return next(new ForbiddenError('...'));  
  
    next();  
  };  
}
```

Listado 6: Middleware de permisos utilizado para proteger rutas.

- No se puede iniciar sesión con una cuenta de usuario inactiva (listado 7).

```
async function login(data, req) {  
  ...  
  const user = await UserRepository.findByEmail(...);  
  ...  
  if (user.status === 'INACTIVE') throw new AuthError('...', 401);  
  ...  
}
```

Listado 7: Comprobación de si el usuario está inactivo en el servicio de inicio de sesión.

- Se debe iniciar sesión para acceder a todas las funcionalidades de la aplicación, menos la vista de inicio de sesión (listado 8).

```
function requireAuth(req, res, next) {  
  if (!req.user)  
    return next(new ForbiddenError());  
  
  next();  
}
```

Listado 8: Middleware de autenticación utilizado para proteger rutas.

Gestión de roles y permisos

El acceso a la aplicación se gestiona mediante el **modelo RBAC**, o *role-based access control*. En un sistema RBAC, cada cuenta de usuario tiene asignado uno o varios roles, determinados por el administrativo que creó la cuenta. A su vez, cada uno de estos roles tiene un conjunto de permisos asociados. Por ejemplo, el rol de administrativo posee el permiso para crear usuarios.

La aplicación cuenta con dos roles, administrativo y cardiólogo, cuyos permisos se mantuvieron relativamente flexibles. Esto se debe a que según el feedback recibido durante el desarrollo del proyecto, en muchos entornos hospitalarios todos o la mayoría

de los usuarios suelen tener permisos para todas las funcionalidades del sistema. Por tanto, tanto el rol de administrativo como el de cardiólogo comparten todos los permisos disponibles del sistema, salvo las operaciones de crear usuario, y crear y modificar agenda, reservadas para perfiles administrativos.

La implementación del middleware que se encarga de comprobar los permisos del usuario se ve en el listado 6, y algunos ejemplos de su uso en la definición de las rutas en el listado 9.

```
router.post('/', requirePermission("user:create"), ...);  
router.get('/', requirePermission("user:read"), ...);  
router.put('/:uuid', requirePermission("user:update"), ...));
```

Listado 9: Ejemplos de uso del middleware de permisos en las rutas de usuario.

3.4.2. Módulo de Paciente

...

Funcionalidades

...

Modelo de datos

Los atributos que tiene la entidad de usuario, así como las relaciones concretas que tiene con el resto de entidades se muestran en la tabla 10 y la tabla 11.

Campo	Tipo	Descripción
id	INTEGER	Identificador interno del paciente
uuid	UUID	Identificador público único del paciente
sex	ENUM("MALE", "FEMALE")	Sexo biológico del paciente
name	STRING(50)	Nombre del paciente
surname	STRING(60)	Apellidos del paciente
email	STRING(100)	Correo electrónico del paciente
phone	STRING(20)	Número de teléfono del paciente
addressLine1	STRING(150)	Primera línea de la dirección del paciente
addressLine2	STRING(150)	Segunda línea de la dirección del paciente
dni	STRING(20)	Documento nacional de identidad del paciente
dateOfBirth	DATEONLY	Fecha de nacimiento del paciente
nhc	STRING(20)	Número de historia clínica del paciente
status	ENUM("ACTIVE", "INACTIVE")	Estado del paciente
createdAt	DATE	Fecha de creación del registro
updatedAt	DATE	Fecha de última actualización del registro

Tabla 10: Modelo de datos de la entidad Paciente.

Entidad relacionada	Cardinalidad
Cita	1 → N
Diagnóstico	1 → N
Tratamiento	1 → N
Flujo de paciente	1 → 1
Registro de auditoría	1 → N

Tabla 11: Relaciones de la entidad Paciente con el resto de entidades del sistema.

En un entorno hospitalario real, un profesional sanitario debería de poder estar en varias agendas, pero este comportamiento se simplificó a una sola agenda por usuario en consideración al alcance del proyecto.

Lógica de negocio

...

3.4.3. Módulo de Agenda

...

Funcionalidades

...

Modelo de datos

Campo	Tipo	Descripción
id	INTEGER	Identificador interno de la agenda
uuid	UUID	Identificador público único de la agenda
name	STRING(50)	Nombre de la agenda
status	ENUM("ACTIVE", "INACTIVE")	Estado de la agenda dentro del sistema
createdAt	DATE	Fecha de creación del registro
updatedAt	DATE	Fecha de última actualización del registro

Tabla 12: Modelo de datos de la entidad Agenda.

Entidad relacionada	Cardinalidad
Periodo de agenda	1 → N
Usuario	1 → N

Tabla 13: Relaciones de la entidad Agenda con el resto de entidades del sistema.

Campo	Tipo	Descripción
id	INTEGER	Identificador interno del periodo de agenda
uuid	UUID	Identificador público único del periodo de agenda
agendaId	INTEGER	Referencia a la agenda asociada
openingDate	DATEONLY	Fecha de apertura del periodo
closingDate	DATEONLY	Fecha de cierre del periodo
agendaStatus	ENUM("OPEN", "CLOSED", "CANCELLED")	Estado funcional del periodo de agenda
status	ENUM("ACTIVE", "INACTIVE")	Estado del registro dentro del sistema
createdAt	DATE	Fecha de creación del registro
updatedAt	DATE	Fecha de última actualización del registro

Tabla 14: Modelo de datos de la entidad Periodo de Agenda.

Entidad relacionada	Cardinalidad
Agenda	N → 1

Tabla 15: Relaciones de la entidad Periodo de Agenda con el resto de entidades del sistema.

Lógica de negocio

...

3.4.4. Módulo de Diagnóstico

...

Funcionalidades

...

Modelo de datos

Campo	Tipo	Descripción
id	INTEGER	Identificador interno del diagnóstico
uuid	UUID	Identificador público único del diagnóstico
patientId	INTEGER	Referencia al paciente asociado
appointmentId	INTEGER	Referencia a la cita asociada
name	STRING(100)	Nombre del diagnóstico
description	TEXT	Descripción detallada del diagnóstico
notes	TEXT	Notas adicionales sobre el diagnóstico
severity	ENUM("LOW", "MODERATE", "HIGH", "CRITICAL")	Nivel de severidad del diagnóstico
clinicalStatus	ENUM("ACTIVE", "RESOLVED", "CHRONIC", "RULED_OUT")	Estado clínico del diagnóstico
status	ENUM("VALID", "VOID", "ENTERED_IN_ERROR")	Estado administrativo del diagnóstico
diagnosedAt	DATE	Fecha y hora del diagnóstico
resolvedAt	DATE	Fecha y hora de resolución del diagnóstico
createdAt	DATE	Fecha de creación del registro
updatedAt	DATE	Fecha de última actualización del registro

Tabla 16: Modelo de datos de la entidad Diagnóstico.

Entidad relacionada	Cardinalidad
Paciente	N → 1
Cita	N → 1
Usuario	N → N
Tratamiento	1 → N
Documento clínico	1 → N

Tabla 17: Relaciones de la entidad Diagnóstico con el resto de entidades del sistema.

Campo	Tipo	Descripción
id	INTEGER	Identificador interno de la asignación
diagnosisId	INTEGER	Referencia al diagnóstico asociado
userId	INTEGER	Referencia al usuario asociado
role	ENUM("AUTHOR", "REVIEWER", "VALIDATOR", "CONTRIBUTOR")	Rol del usuario dentro del diagnóstico
assignedAt	DATE	Fecha de asignación del usuario al diagnóstico
createdAt	DATE	Fecha de creación del registro
updatedAt	DATE	Fecha de última actualización del registro

Tabla 18: Modelo de datos de la entidad Diagnóstico-Usuario.

Entidad relacionada	Cardinalidad
Diagnóstico	$N \rightarrow 1$
Usuario	$N \rightarrow 1$

Tabla 19: Relaciones de la entidad Diagnóstico-Usuario con el resto de entidades del sistema.

Lógica de negocio

...

3.4.5. Módulo de Tratamiento

...

Funcionalidades

...

Modelo de datos

Campo	Tipo	Descripción
id	INTEGER	Identificador interno del tratamiento
uuid	UUID	Identificador público único del tratamiento
patientId	INTEGER	Referencia al paciente asociado
appointmentId	INTEGER	Referencia a la cita asociada
diagnosisId	INTEGER	Referencia al diagnóstico asociado
name	STRING(100)	Nombre del tratamiento
description	TEXT	Descripción detallada del tratamiento
notes	TEXT	Notas adicionales sobre el tratamiento
duration	STRING(80)	Duración estimada o planificada del tratamiento
clinicalStatus	ENUM("PLANNED", "ONGOING", "GIVEN", "COMPLETED", "DISCONTINUED")	Estado clínico del tratamiento
status	ENUM("VALID", "VOID", "ENTERED_IN_ERROR")	Estado administrativo del tratamiento
resolvedAt	DATE	Fecha y hora de finalización del tratamiento
createdAt	DATE	Fecha de creación del registro
updatedAt	DATE	Fecha de última actualización del registro

Tabla 20: Modelo de datos de la entidad Tratamiento.

Entidad relacionada	Cardinalidad
Paciente	$N \rightarrow 1$
Cita	$N \rightarrow 1$
Diagnóstico	$N \rightarrow 1$
Usuario	$N \rightarrow N$
Documento clínico	$1 \rightarrow N$

Tabla 21: Relaciones de la entidad Tratamiento con el resto de entidades del sistema.

Campo	Tipo	Descripción
id	INTEGER	Identificador interno de la asignación
treatmentId	INTEGER	Referencia al tratamiento asociado
userId	INTEGER	Referencia al usuario asociado
role	ENUM("AUTHOR", "REVIEWER", "VALIDATOR", "CONTRIBUTOR")	Rol del usuario dentro del tratamiento
assignedAt	DATE	Fecha de asignación del usuario al tratamiento
createdAt	DATE	Fecha de creación del registro
updatedAt	DATE	Fecha de última actualización del registro

Tabla 22: Modelo de datos de la entidad Tratamiento-Usuario.

Entidad relacionada	Cardinalidad
Tratamiento	$N \rightarrow 1$
Usuario	$N \rightarrow 1$

Tabla 23: Relaciones de la entidad Tratamiento-Usuario con el resto de entidades del sistema.

3.4.5.1. Lógica de negocio

...

3.4.6. Módulo de Documento Clínico

...

Funcionalidades

...

Modelo de datos

Campo	Tipo	Descripción
id	INTEGER	Identificador interno del documento clínico
uuid	UUID	Identificador público único del documento clínico
userId	INTEGER	Referencia al usuario creador del documento
title	STRING(100)	Título del documento clínico
documentType	ENUM("CLINICAL_SUMMARY", "PROGRESS_NOTE", "CONSULT_NOTE", "EXTERNAL_FILE", "OTHER")	Tipo de documento clínico
content	JSONB	Contenido estructurado del documento clínico
status	ENUM("DRAFTED", "FINAL", "AMENDED", "ARCHIVED", "VOID")	Estado del documento clínico
createdAt	DATE	Fecha de creación del registro
updatedAt	DATE	Fecha de última actualización del registro

Tabla 24: Modelo de datos de la entidad Documento Clínico.

Entidad relacionada	Cardinalidad
Usuario	N → N
Archivo clínico	N → N
Diagnóstico	N → N
Tratamiento	N → N

Tabla 25: Relaciones de la entidad Documento Clínico con el resto de entidades del sistema.

Campo	Tipo	Descripción
id	INTEGER	Identificador interno del archivo clínico
userId	INTEGER	Referencia al usuario propietario del archivo
fileName	STRING(100)	Nombre original del archivo
storageKey	UUID	Clave única de almacenamiento del archivo
mimeType	STRING(50)	Tipo MIME del archivo
fileSize	BIGINT	Tamaño del archivo en bytes
status	ENUM("ACTIVE", "INACTIVE", "DELETED")	Estado del archivo clínico
createdAt	DATE	Fecha de creación del registro
updatedAt	DATE	Fecha de última actualización del registro

Tabla 26: Modelo de datos de la entidad Archivo Clínico.

Entidad relacionada	Cardinalidad
Usuario	$N \rightarrow 1$
Documento clínico	$N \rightarrow N$

Tabla 27: Relaciones de la entidad Archivo Clínico con el resto de entidades del sistema.

Campo	Tipo	Descripción
id	INTEGER	Identificador interno de la asignación
clinicalDocumentId	INTEGER	Referencia al documento clínico asociado
userId	INTEGER	Referencia al usuario asociado
role	ENUM("AUTHOR", "REVIEWER", "VALIDATOR", "CONTRIBUTOR", "UPLOADER")	Rol del usuario dentro del documento clínico
createdAt	DATE	Fecha de creación del registro
updatedAt	DATE	Fecha de última actualización del registro

Tabla 28: Modelo de datos de la entidad Documento Clínico-Usuario.

Entidad relacionada	Cardinalidad
Documento clínico	$N \rightarrow 1$
Usuario	$N \rightarrow 1$

Tabla 29: Relaciones de la entidad Documento Clínico-Usuario con el resto de entidades del sistema.

Lógica de negocio

...

3.4.7. Módulo de Cita

...

Funcionalidades

...

Modelo de datos

Campo	Tipo	Descripción
id	INTEGER	Identificador interno de la cita
uuid	UUID	Identificador público único de la cita
userId	INTEGER	Referencia al profesional sanitario asociado
patientId	INTEGER	Referencia al paciente asociado
reason	STRING(255)	Motivo principal de la cita
notes	TEXT	Notas adicionales sobre la cita
startTime	DATE	Fecha y hora de inicio de la cita
endTime	DATE	Fecha y hora de finalización de la cita
location	STRING(100)	Ubicación física o virtual de la cita
type	ENUM("IN_PERSON", "VIRTUAL")	Tipo de cita médica
status	ENUM("SCHEDULED", "COMPLETED", "CANCELLED", "NO_SHOW", "CHECKED_IN")	Estado de la cita dentro del sistema
createdAt	DATE	Fecha de creación del registro
updatedAt	DATE	Fecha de última actualización del registro

Tabla 30: Modelo de datos de la entidad Cita.

Entidad relacionada	Cardinalidad
Usuario	$N \rightarrow 1$
Paciente	$N \rightarrow 1$
Diagnóstico	$1 \rightarrow N$
Tratamiento	$1 \rightarrow N$

Tabla 31: Relaciones de la entidad Cita con el resto de entidades del sistema.

Lógica de negocio

...

3.4.8. Módulo de Flujo de Paciente

Funcionalidades

...

Modelo de datos

Campo	Tipo	Descripción
id	INTEGER	Identificador interno del flujo de paciente
uuid	UUID	Identificador público único del flujo de paciente
patientId	INTEGER	Referencia al paciente asociado
createdAt	DATE	Fecha de creación del registro
updatedAt	DATE	Fecha de última actualización del registro

Tabla 32: Modelo de datos de la entidad Flujo de Paciente.

Entidad relacionada	Cardinalidad
Paciente	1 → 1
Evento de flujo	1 → N

Tabla 33: Relaciones de la entidad Flujo de Paciente con el resto de entidades del sistema.

Campo	Tipo	Descripción
id	INTEGER	Identificador interno del evento de flujo
patientFlowId	INTEGER	Referencia al flujo de paciente asociado
date	DATE	Fecha y hora del evento
title	STRING(100)	Título descriptivo del evento
type	ENUM("APPOINTMENT", "DIAGNOSIS", "TREATMENT", "CLINICAL_DOCUMENT", "PATIENT", "OTHER")	Tipo de entidad representada en el flujo
role	ENUM("PRIMARY", "SECONDARY")	Rol estructural del evento dentro del flujo
parentEventId	INTEGER	Referencia al evento padre asociado
entityId	INTEGER	Referencia a la entidad relacionada con el evento
positionX	FLOAT	Posición horizontal del evento dentro del flujo visual
positionY	FLOAT	Posición vertical del evento dentro del flujo visual
createdAt	DATE	Fecha de creación del registro
updatedAt	DATE	Fecha de última actualización del registro

Tabla 34: Modelo de datos de la entidad Evento de Flujo.

Entidad relacionada	Cardinalidad
Flujo de paciente	$N \rightarrow 1$
Evento de flujo	$N \rightarrow 1$
Evento de flujo (hijo)	$1 \rightarrow N$

Tabla 35: Relaciones de la entidad Evento de Flujo con el resto de entidades del sistema.

Lógica de negocio

...

3.4.9. Módulo de Dashboard

...

Funcionalidades

...

Modelo de datos

Campo	Tipo	Descripción
id	INTEGER	Identificador interno del dashboard
uuid	UUID	Identificador público único del dashboard
userId	INTEGER	Referencia al usuario propietario del dashboard
createdAt	DATE	Fecha de creación del registro
updatedAt	DATE	Fecha de última actualización del registro

Tabla 36: Modelo de datos de la entidad Dashboard de Usuario.

Entidad relacionada	Cardinalidad
Usuario	1 → 1
Componente de dashboard	1 → N

Tabla 37: Relaciones de la entidad Dashboard de Usuario con el resto de entidades del sistema.

Campo	Tipo	Descripción
id	INTEGER	Identificador interno del componente
dashboardId	INTEGER	Referencia al dashboard asociado
title	STRING(100)	Título descriptivo del componente
type	ENUM("KPI", "LINE_CHART", "BAR_CHART", "PIE_CHART", "LIST")	Tipo de visualización del componente
config	JSON	Configuración estructurada del componente
position	JSON	Posición y dimensiones del componente dentro del dashboard
source	ENUM("SYSTEM", "USER")	Origen del componente dentro del sistema
createdAt	DATE	Fecha de creación del registro
updatedAt	DATE	Fecha de última actualización del registro

Tabla 38: Modelo de datos de la entidad Componente de Dashboard.

Entidad relacionada	Cardinalidad
Dashboard de usuario	N → 1

Tabla 39: Relaciones de la entidad Componente de Dashboard con el resto de entidades del sistema.

Lógica de negocio

...

3.4.10. Módulo de Auditoría

...

Funcionalidades

...

Modelo de datos

Campo	Tipo	Descripción
id	INTEGER	Identificador interno del registro de auditoría
uuid	UUID	Identificador público único del registro de auditoría
action	ENUM("CREATED", "STATUS_CHANGED", "UPDATED", "DELETED", "CLINICAL_STATUS_CHANGED")	Acción registrada dentro del sistema
entityType	ENUM("DIAGNOSIS", "TREATMENT", "APPOINTMENT", "CLINICAL_DOCUMENT", "PATIENT")	Tipo de entidad afectada por la acción
entityId	INTEGER	Referencia a la entidad afectada
userId	INTEGER	Referencia al usuario que realizó la acción
patientId	INTEGER	Referencia al paciente asociado al evento
meta	JSON	Información adicional asociada al evento de auditoría
createdAt	DATE	Fecha de creación del registro
updatedAt	DATE	Fecha de última actualización del registro

Tabla 40: Modelo de datos de la entidad Registro de Auditoría.

Entidad relacionada	Cardinalidad
Usuario	N → 1
Paciente	N → 1

Tabla 41: Relaciones de la entidad Registro de Auditoría con el resto de entidades del sistema.

Lógica de negocio

...

3.5. Detalles generales de implementación

3.6. Pruebas y validación

la mayoría no vale la pena testarlo, son solo componentes, orquestación, cosas básicas
los tests de componente gordo estan centrados en logica

3.7. Otros paquetes útiles en Typst

Hay diversos paquetes disponibles que pueden ser útiles para un TFG: para dibujar diagramas generales ([cetz](#), inspirado en el paquete TikZ de LaTeX), diagramas conmutativos, de flujo o autómatas ([fletcher](#)), visualización de datos ([lilaq](#)), árboles sencillos ([tdtr](#)), árboles de demostración ([curryst](#)), diagramas de Gantt ([timeliney](#)), diagramas UML ([pintorita](#)), etc.

Capítulo 4

Conclusiones y Trabajo Futuro

Conclusiones del trabajo y líneas de trabajo futuro.

Antes de la entrega de actas de cada convocatoria, en el plazo que se indica en el calendario de los trabajos de fin de grado, el estudiante entregará en el Campus Virtual la versión final de la memoria en PDF.

4.1. Explicaciones adicionales sobre el uso de esta plantilla

Esta plantilla está escrita en **Typst**. El **tutorial oficial** y la **documentación** del lenguaje están disponibles en los enlaces anteriores. Typst se puede utilizar a través de

- la **extensión Tinymist** para Visual Studio Code (u otros IDE),
- su **aplicación web**, o
- usando directamente **su compilador** con los comandos `typst compile memoria.typ` o `typst watch memoria.typ`.

Si quieres cambiar el **estilo del título** de los capítulos del documento, edita el fichero `tfxfdi/plantilla.typ` y edita la regla `show rule heading` de la función `cuerpo`.

Si prefieres que no haya espacio entre párrafos y que la primera línea de cada uno aparezca indentada, cambia la configuración con

```
#set par(first-line-indent: 1em, spacing: 0.65em)
```

4.2. Citar

Esta plantilla se ha elaborado a partir de TFGTeXiS, a su vez elaborada a partir de la plantilla de TeXiS¹, creada por Marco Antonio y Pedro Pablo Gómez Martín para escribir su tesis doctoral.

En el estado de la cuestión es donde aparecen gran parte de las referencias bibliográficas del trabajo. Una de las formas más cómodas de gestionar la bibliografía en

¹<http://gaia.fdi.ucm.es/research/texis/>

Typst es utilizando BibTeX. Las entradas bibliográficas deben estar en un fichero con extensión .bib (con esta plantilla se proporciona el fichero bibliografía.bib, donde están las entradas referenciadas más abajo). Cada entrada bibliográfica tiene una clave que permite referenciarla desde cualquier parte del texto con los comandos @typstdoc ([19]) o #cite(<normativa>) ([20]), que tiene más argumentos para configurar diversas opciones. Es posible citar más de una fuente, como por ejemplo [19,21].

Después, Typst se ocupa de rellenar la sección de bibliografía con las entradas **que hayan sido citadas** (es decir, no con todas las entradas que hay en el .bib, sino sólo con aquellas que se hayan citado en alguna parte del texto).

La sintaxis @etiqueta es la misma que se utiliza para hacer referencia a secciones del texto, tablas, figuras, etc., aunque el comando equivalente es ref en lugar de cite. En este caso, debes colocar una etiqueta <etiqueta> tras el elemento al que quieras hacer referencia (como en el encabezado de este capítulo 2).

Introduction

Introduction to the subject area. This chapter contains the translation of Chapter 1.

Conclusions and future work

Conclusions and future lines of work. This chapter contains the translation of Chapter 4.

Bibliografía

- [1] J. Tummers, B. Tekinerdogan, H. Tobi, C. Catal, y B. Schalk, «Obstacles and features of health information systems: A systematic literature review», *Computers in Biology and Medicine*, vol. 137, p. 104785, 2021, doi: <https://doi.org/10.1016/j.combiomed.2021.104785>.
- [2] International Organization for Standardization, «Electronic health records explained». [En línea]. Disponible en: <https://www.iso.org/healthcare/electronic-health-records>
- [3] M. Al Ani, G. Garas, J. Hollingshead, D. Cheetham, T. Athanasiou, y V. Patel, «Which Electronic Health Record System Should We Use? A Systematic Review», *Medical Principles and Practice*, vol. 31, n.º 4, pp. 342-351, 2022, doi: [10.1159/000525135](https://doi.org/10.1159/000525135).
- [4] L. Ahmadian, N. Dorosti, R. Khajouei, y S. H. Gohari, «Challenges of Using Hospital Information Systems by Nurses: Comparing Academic and Non-Academic Hospitals», *Electronic Physician*, vol. 9, n.º 6, pp. 4625-4630, 2017, doi: [10.19082/4625](https://doi.org/10.19082/4625).
- [5] IBM, «What is Business Intelligence (BI)?». [En línea]. Disponible en: <https://www.ibm.com/think/topics/business-intelligence>
- [6] V. L. West, D. Borland, y W. E. Hammond, «Innovative information visualization of electronic health record data: a systematic review», *Journal of the American Medical Informatics Association*, vol. 22, n.º 2, pp. 330-339, 2015, doi: [10.1136/amiajnl-2014-002955](https://doi.org/10.1136/amiajnl-2014-002955).
- [7] G. Lindemulder y M. Kosinski, «What is interoperability?». [En línea]. Disponible en: <https://www.ibm.com/think/topics/interoperability>
- [8] R. S. P. Maciel, P. H. Valle, K. S. Santos, y E. Y. Nakagawa, «Systems Interoperability Types: A Tertiary Study». [En línea]. Disponible en: <https://arxiv.org/abs/2310.19999>
- [9] M. Lehne, J. Sass, A. Essenwanger, J. Schepers, y S. Thun, «Why digital medicine depends on interoperability», *npj Digital Medicine*, vol. 2, n.º 1, p. 79, 2019, doi: [10.1038/s41746-019-0158-1](https://doi.org/10.1038/s41746-019-0158-1).

-
- [10] F. Mollerus, C. Lynch, y H. Bruining, «Data interoperability for a systems approach to developmental conditions», *Neuroscience & Biobehavioral Reviews*, vol. 176, p. 106245, 2025, doi: <https://doi.org/10.1016/j.neubiorev.2025.106245>.
 - [11] D. W. Bates y L. Samal, «Interoperability: What Is It, How Can We Make It Work for Clinicians, and How Should We Measure It in the Future?», *Health Services Research*, vol. 53, n.º 5, pp. 3270-3277, 2018, doi: [10.1111/1475-6773.12852](https://doi.org/10.1111/1475-6773.12852).
 - [12] Feature-Sliced Design, «Feature-Sliced Design».
 - [13] A. Alickovic, «Bulletproof React».
 - [14] D. Jonauskaite y C. Mohr, «Do we feel colours? A systematic review of 128 years of psychological research linking colours and emotions», *Psychon. Bull. Rev.*, vol. 32, n.º 4, pp. 1457-1486, ago. 2025.
 - [15] H.-H. Wang, «Few Guesses, More Success: 4 Principles to Reduce Cognitive Load in Forms». Accedido: 9 de mayo de 2026. [En línea]. Disponible en: <https://www.nngroup.com/articles/4-principles-reduce-cognitive-load/>
 - [16] E. Scott, «Form Field Usability: Avoid Extensive Multicolumn Layouts (16% Make This Form Usability Mistake)». Accedido: 9 de mayo de 2026. [En línea]. Disponible en: <https://baymard.com/blog/avoid-multi-column-forms>
 - [17] R. Krause, «10 Design Guidelines for Reporting Errors in Forms». Accedido: 9 de mayo de 2026. [En línea]. Disponible en: <https://www.nngroup.com/articles/errors-forms-design-guidelines/>
 - [18] P. Laubheimer, «Left-Side Vertical Navigation on Desktop: Scalable, Responsive, and Easy to Scan». Accedido: 9 de mayo de 2026. [En línea]. Disponible en: <https://www.nngroup.com/articles/vertical-nav/>
 - [19] Typst developers, «Typst Documentation». Accedido: 3 de febrero de 2026. [En línea]. Disponible en: <https://typst.app/docs/>
 - [20] Facultad de Informática, «Normativa de la asignatura Trabajo de Fin de Grado». Accedido: 3 de febrero de 2026. [En línea]. Disponible en: <https://informatica.ucm.es/file/normativa-tfg-actual>
 - [21] E. W. Dijkstra, «The Humble Programmer», *Edsger Wybe Dijkstra: His Life, Work, and Legacy*, vol. 45. en ACM Books, vol. 45. ACM / Morgan & Claypool, pp. 5-18, 2022. doi: [10.1145/3544585.3544588](https://doi.org/10.1145/3544585.3544588).

Apéndice **A**

Especificación de Requisitos

A.A. Módulo de administración

Requisito	Dar de alta a usuario
Identificador	1.1
Prioridad	Alta
Descripción	Da de alta a un nuevo usuario en la aplicación.
Actor/es	Administrativo
Entrada	• Nombre • Apellidos • Correo electrónico • Contraseña • Agenda • Roles • Teléfono
Salida	• Nuevo usuario con los datos de entrada en la aplicación. • Mensaje indicando el éxito de la operación.
Secuencia de Acciones	1. El actor solicita el formulario para dar de alta a un nuevo usuario; el sistema muestra el formulario correspondiente. 2. El actor rellena los campos del formulario con la información correspondiente. 3. El actor solicita realizar el registro del usuario. 4. El sistema valida los datos. 5. El sistema crea al nuevo usuario y notifica de ello.
Precondición	El actor ha iniciado sesión con una cuenta con rol de administrativo.
Postcondición	La aplicación cuenta con un nuevo usuario válido que puede autenticarse en ella y acceder a distintos servicios según su rol.
Excepciones	• Si ya existe un usuario con el correo introducido, el sistema mostrará un mensaje indicando el error. • Si no se cumplen los criterios de validación para cada campo del formulario, el sistema mostrará un mensaje con el error correspondiente.
Comentarios	N.A.

Tabla A.1: Requisito 1.1 - Dar de alta a usuario.

Requisito	Modificar datos de usuario
Identificador	1.2
Prioridad	Media
Descripción	Modifica los datos de un usuario existente en la aplicación.
Actor/es	Administrativo
Entrada	• Identificador • Nombre • Apellidos • Correo electrónico • Teléfono • Roles
Salida	• Usuario existente modificado según los datos de entrada. • Mensaje indicando el éxito de la operación.
Secuencia de Acciones	1. El actor solicita el formulario para modificar a un usuario existente; el sistema muestra el formulario correspondiente. 2. El actor modifica los valores deseados. 3. El actor solicita realizar la modificación de los datos del usuario. 4. El sistema valida los datos. 5. El sistema modifica los datos del usuario y notifica de ello.
Precondición	El actor ha iniciado sesión con una cuenta con rol de administrativo.
Postcondición	La aplicación cuenta con un usuario que ya existía, pero que posee uno o más atributos con valor distinto a los que tenía antes de la operación.
Excepciones	• Si ya existe un usuario con el correo introducido, el sistema mostrará un mensaje indicando el error. • Si no se cumplen los criterios de validación para cada campo del formulario, el sistema mostrará un mensaje con el error correspondiente.
Comentarios	N.A.

Tabla A.2: Requisito 1.2 - Modificar datos de usuario.

Requisito	Dar de baja a usuario
Identificador	1.3
Prioridad	Alta
Descripción	Da de baja a un usuario existente en la aplicación.
Actor/es	Administrativo
Entrada	Identificador del usuario
Salida	- Usuario existente dado de baja. - Mensaje indicando el éxito de la operación.
Secuencia de Acciones	1. El actor solicita dar de baja a un usuario existente; el sistema da de baja al usuario y notifica de ello.
Precondición	El actor ha iniciado sesión con una cuenta con rol de administrativo.
Postcondición	La aplicación cuenta con un usuario que ya existía, que ahora está dado de baja, y por tanto no es posible autenticarse con él.
Excepciones	Si el usuario tiene dependencias activas en la base de datos, el sistema mostrará un mensaje con el error correspondiente.
Comentarios	No elimina al usuario, sino que deja su cuenta inactiva.

Tabla A.3: Requisito 1.3 - Dar de baja a usuario.

Requisito	Ver usuario
Identificador	1.4
Prioridad	Media
Descripción	Muestra la información de un usuario.
Actor/es	Administrativo
Entrada	Identificador del usuario
Salida	Todos los atributos del usuario.
Secuencia de Acciones	1. El actor solicita ver la vista con la información del usuario; el sistema muestra la vista con la información correspondiente.
Precondición	El actor ha iniciado sesión con una cuenta con rol de administrativo.
Postcondición	La información del usuario solicitado queda mostrada en pantalla.
Excepciones	N.A.
Comentarios	El administrativo tiene acceso a toda la información del sistema, por lo que incluye no solo la información de perfil, sino también todos los datos vinculados a cada usuario.

Tabla A.4: Requisito 1.4 - Ver usuario.

Requisito	Listar usuarios
Identificador	1.5
Prioridad	Media
Descripción	Lista todos los usuarios registrados, con la posibilidad de aplicar filtros por atributos, y ordenar los resultados por atributos de forma ascendente o descendente.
Actor/es	Administrativo
Entrada	• Atributos opcionales por los que filtrar y/o ordenar • Sentido de ordenación
Salida	• Lista de todos los usuarios tras aplicar posibles filtros y ordenamientos.
Secuencia de Acciones	1. El actor solicita la vista para listar usuarios; el sistema devuelve la vista correspondiente con la lista de usuarios. 2. El actor escoge los filtros y orden que quiere aplicar a los resultados, que pueden ser ninguno. 3. El actor solicita actualizar la lista de usuarios. 4. El sistema aplica los filtros y criterios de ordenación. 5. El sistema devuelve la lista de usuarios solicitada.
Precondición	El actor ha iniciado sesión con una cuenta con rol de administrativo.
Postcondición	La lista de usuarios con los filtros y criterios de ordenación solicitados queda mostrada en pantalla.
Excepciones	N.A.
Comentarios	N.A.

Tabla A.5: Requisito 1.5 - Listar usuarios.

A.B. Módulo de usuarios

Requisito	Iniciar sesión
Identifica- dor	2.1
Prioridad	Alta
Descripción	Permite a un usuario autenticarse en la aplicación mediante sus credenciales.
Actor/es	Administrativo, cardiólogo
Entrada	• Correo electrónico • Contraseña
Salida	• La vista de la aplicación correspondiente al usuario y las funcionalidades según su rol.
Secuencia de Acciones	1. El actor solicita el formulario para iniciar sesión; el sistema muestra el formulario correspondiente. 2. El actor rellena los campos del formulario con la información correspondiente. 3. El actor solicita realizar el inicio de sesión. 4. El sistema valida los datos. 5. El sistema autentica al usuario con las credenciales introducidas.
Precondi- ción	El actor no está autenticado en la aplicación.
Postcondi- ción	El actor está autenticado en la aplicación, y puede ver la vista y funcionalidades que le corresponden según su usuario.
Excepciones	• Si el correo electrónico y contraseña introducidos no corresponden con ningún usuario existente en la base de datos, se mostrará un mensaje con el error correspondiente. • Si ya hay una sesión iniciada con las credenciales introducidas en otra pestaña o dispositivo, se mostrará un mensaje de error.
Comenta- rios	N.A.

Tabla A.6: Requisito 2.1 - Iniciar sesión.

Requisito	Cerrar sesión
Identificador	2.2
Prioridad	Alta
Descripción	Permite a un usuario cerrar sesión en la aplicación.
Actor/es	Administrativo, cardiólogo.
Entrada	N.A.
Salida	La vista de la aplicación sin ningún usuario autenticado.
Secuencia de Acciones	1. El actor solicita cerrar sesión; el sistema cierra la sesión del usuario.
Precondición	El actor está autenticado en la aplicación.
Postcondición	El actor no está autenticado en la aplicación, y puede ver la vista y funcionalidades que corresponden a la aplicación sin ninguna sesión activa.
Excepciones	N.A.
Comentarios	N.A.

Tabla A.7: Requisito 2.2 - Cerrar sesión.

Requisito	Restablecer contraseña
Identificador	2.3
Prioridad	Baja
Descripción	Permite a un usuario restablecer su contraseña.
Actor/es	Administrativo, cardiólogo
Entrada	• Contraseña antigua • Nueva contraseña
Salida	• Usuario existente tiene una contraseña nueva. • Mensaje indicando el éxito de la operación.
Secuencia de Acciones	1. El actor solicita el formulario para restablecer su contraseña; el sistema muestra el formulario correspondiente. 2. El actor rellena los campos del formulario con la información correspondiente. 3. El actor solicita realizar el restablecimiento de su contraseña 4. El sistema valida los datos. 5. El sistema restablece la contraseña y notifica de ello.
Precondición	El actor no está autenticado en la aplicación.
Postcondición	La aplicación cuenta con un usuario que ya existía, pero que posee una contraseña distinta a la que tenía previamente.
Excepciones	• Si el correo electrónico introducido no corresponde con ningún usuario existente en la base de datos, se mostrará un mensaje con el error correspondiente. • Si la nueva contraseña no cumple con los criterios de validación impuestos, se mostrará un mensaje con el error correspondiente • Si ya hay una sesión iniciada con el correo electrónico introducido en otra pestaña o dispositivo, se mostrará un mensaje de error.
Comentarios	N.A.

Tabla A.8: Requisito 2.3 - Restablecer contraseña.

Requisito	Modificar datos del perfil
Identificador	2.5
Prioridad	Baja
Descripción	Modifica los datos de la cuenta del actor.
Actor/es	Administrativo, cardiólogo
Entrada	• Nombre • Apellidos • Correo • Teléfono • Roles
Salida	• Cuenta del actor modificada según los datos de entrada. • Mensaje indicando el éxito de la operación.
Secuencia de Acciones	1. El actor solicita el formulario para modificar su perfil; el sistema muestra el formulario correspondiente. 2. El actor modifica los valores deseados. 3. El actor solicita realizar la modificación de los datos del perfil. 4. El sistema valida los datos. 5. El sistema modifica los datos del perfil y notifica de ello.
Precondición	El actor está autenticado en la aplicación.
Postcondición	La aplicación cuenta con un usuario que ya existía, pero que posee uno o más atributos con valor distinto a los que tenía antes de la operación.
Excepciones	• Si ya existe un usuario con el correo introducido, el sistema mostrará un mensaje indicando el error. • Si no se cumplen los criterios de validación para cada campo del formulario, el sistema mostrará un mensaje con el error correspondiente.
Comentarios	N.A.

Tabla A.9: Requisito 2.5 - Modificar datos del perfil.

Requisito	Aplicar opción de accesibilidad
Identificador	2.6
Prioridad	Media
Descripción	Permite a los usuarios aplicar diversas opciones de accesibilidad para mejorar su experiencia en base a las necesidades que puedan tener.
Actor/es	Administrativo, cardiólogo
Entrada	Tipo de opción de accesibilidad
Salida	Aspecto de la página web del actor modificado según la opción escogida.
Secuencia de Acciones	1. El actor solicita la vista con las opciones de accesibilidad; el sistema muestra la vista correspondiente. 2. El actor selecciona la opción de accesibilidad que quiere. 3. El sistema aplica la opción de accesibilidad.
Precondición	El actor está autenticado en la aplicación.
Postcondición	La aplicación cuenta con un usuario que ya existía, pero que posee uno o más atributos con valor distinto a los que tenía antes de la operación.
Excepciones	N.A.
Comentarios	Las opciones incluyen: poder cambiar el contraste de color.

Tabla A.10: Requisito 2.6 - Aplicar opción de accesibilidad.

A.C. Módulo de pacientes

Requisito	Alta de paciente
Identificador	3.1
Prioridad	Alta
Descripción	Da de alta a un nuevo paciente en la aplicación
Actor/es	Administrativo, cardiólogo
Entrada	• Nombre • Apellidos • Fecha de nacimiento • Sexo • DNI • Correo • Teléfono • Dirección
Salida	• Nuevo paciente con los datos de entrada en la aplicación. • NHC (número de historia clínica) del paciente. • Mensaje indicando el éxito de la operación.
Secuencia de Acciones	1. El actor solicita el formulario para dar de alta a un nuevo paciente; el sistema muestra el formulario correspondiente. 2. El actor rellena los campos del formulario. 3. El actor solicita realizar el registro del paciente. 4. El sistema valida los datos. 5. El sistema crea al nuevo paciente y notifica de ello.
Precondición	El actor está autenticado en la aplicación.
Postcondición	La aplicación cuenta con un nuevo paciente válido.
Excepciones	• Si ya existe un paciente cuyo DNI coincida con el introducido, se mostrará un mensaje de aviso por posible duplicado. • Si no se cumplen los criterios de validación para cada campo del formulario, el sistema mostrará un mensaje con el error correspondiente.
Comentarios	N.A.

Tabla A.11: Requisito 3.1 - Alta de paciente.

Requisito	Modificar datos de paciente
Identificador	3.2
Prioridad	Media
Descripción	Modifica los datos de un paciente existente en la aplicación
Actor/es	Administrativo, cardiólogo
Entrada	• Identificador • Nombre • Apellidos • Correo • Teléfono • Dirección
Salida	• Paciente existente modificado según los datos de entrada. • Mensaje indicando el éxito de la operación.
Secuencia de Acciones	1. El actor solicita el formulario para modificar a un paciente existente; el sistema muestra el formulario correspondiente. 2. El actor modifica los valores deseados. 3. El actor solicita realizar la modificación de los datos del usuario. 4. El sistema valida los datos. 5. El sistema modifica los datos del paciente y notifica de ello.
Precondición	El actor está autenticado en la aplicación.
Postcondición	La aplicación cuenta con un paciente que ya existía, pero que posee uno o más atributos con valor distinto a los que tenía antes de la operación.
Excepciones	• Si ya existe un paciente cuyo DNI coincida con el introducido, se mostrará un mensaje de aviso por posible duplicado. • Si no se cumplen los criterios de validación para cada campo del formulario, el sistema mostrará un mensaje con el error correspondiente.
Comentarios	N.A.

Tabla A.12: Requisito 3.2 - Modificar datos de paciente.

Requisito	Dar de baja a paciente
Identificador	3.3
Prioridad	Baja
Descripción	Da de baja a un paciente existente en la aplicación.
Actor/es	Administrativo, cardiólogo
Entrada	Identificador del paciente
Salida	- Paciente existente dado de baja. - Mensaje indicando el éxito de la operación.
Secuencia de Acciones	1. El actor solicita dar de baja a un paciente existente; el sistema da de baja al paciente y notifica de ello.
Precondición	El actor está autenticado en la aplicación.
Postcondición	La aplicación cuenta con un paciente que ya existía, que ahora está dado de baja y por tanto ya no se le pueden asociar nuevas citas, tratamientos, ni diagnósticos.
Excepciones	Si el paciente tiene dependencias activas en la base de datos, el sistema mostrará un mensaje con el error correspondiente.
Comentarios	No elimina al paciente, sino que se mantiene en la base de datos marcado como inactivo.

Tabla A.13: Requisito 3.3 - Dar de baja a paciente.

Requisito	Ver paciente
Identificador	3.4
Prioridad	Media
Descripción	Muestra la información de un paciente.
Actor/es	Administrativo, cardiólogo
Entrada	• Identificador del paciente
Salida	• Todos los atributos del paciente
Secuencia de Acciones	1. El actor solicita ver la vista con la información del paciente; el sistema muestra la vista con la información correspondiente.
Precondición	El actor está autenticado en la aplicación.
Postcondición	La información del paciente solicitado queda mostrada en pantalla.
Excepciones	N.A.
Comentarios	N.A.

Tabla A.14: Requisito 3.4 - Ver paciente.

Requisito	Listar pacientes
Identificador	3.5
Prioridad	Media
Descripción	Lista todos los pacientes, con la posibilidad de aplicar filtros por atributos, y ordenar los resultados por atributos de forma ascendente o descendente.
Actor/es	Administrativo, cardiólogo
Entrada	• Atributos opcionales por los que filtrar y/o ordenar • Sentido de ordenación
Salida	• Lista de todos los pacientes tras aplicar filtros.
Secuencia de Acciones	1. El actor solicita la vista para listar pacientes; el sistema devuelve la vista correspondiente con la lista de usuarios. 2. El actor escoge los filtros y orden que quiere aplicar a los resultados, que pueden ser ninguno. 3. El actor solicita actualizar la lista de pacientes. 4. El sistema aplica los filtros y criterios de ordenación. 5. El sistema devuelve la lista de pacientes solicitada.
Precondición	El actor está autenticado en la aplicación.
Postcondición	La lista de pacientes con los filtros y criterios de ordenación solicitados queda mostrada en pantalla.
Excepciones	N.A.
Comentarios	N.A.

Tabla A.15: Requisito 3.5 - Listar pacientes.

A.D. Módulo de citas

Requisito	Crear cita
Identifica- dor	4.1
Prioridad	Alta
Descripción	Crea una cita en la aplicación.
Actor/es	Administrativo, cardiólogo
Entrada	• Paciente • Usuario • Modalidad • Fecha de inicio • Lugar • Motivo • Notas
Salida	• Nueva cita con los datos de entrada en la aplicación. • Mensaje indicando el éxito de la operación.
Secuencia de Acciones	1. El actor solicita el formulario para crear una nueva cita; el sistema muestra el formulario correspondiente. 2. El actor rellena los campos del formulario con la información correspondiente. 3. El actor solicita realizar la creación de la cita. 4. El sistema valida los datos. 5. El sistema crea la nueva cita y notifica de ello.
Precondi- ción	El actor está autenticado en la aplicación.
Postcondi- ción	La aplicación cuenta con una nueva cita.
Excepciones	• Si no se cumplen los criterios de validación para cada campo del formulario, el sistema mostrará un mensaje con el error correspondiente.
Comenta- rios	N.A.

Tabla A.16: Requisito 4.1 - Crear cita.

Requisito	Modificar datos de cita
Identificador	4.2
Prioridad	Baja
Descripción	Modifica los datos de una cita existente en la aplicación.
Actor/es	Administrativo, cardiólogo
Entrada	• Identificador • Modalidad • Fecha de inicio • Lugar • Motivo • Notas
Salida	• Cita existente modificada según los datos de entrada. • Mensaje indicando el éxito de la operación.
Secuencia de Acciones	1. El actor solicita el formulario para modificar una cita existente; el sistema muestra el formulario correspondiente. 2. El actor modifica los valores deseados. 3. El actor solicita realizar la modificación de los datos de la cita. 4. El sistema valida los datos. 5. El sistema modifica los datos de la cita y notifica de ello.
Precondición	El actor está autenticado en la aplicación.
Postcondición	La aplicación cuenta con una cita que ya existía, pero que posee uno o más atributos con valor distinto a los que tenía antes de la operación.
Excepciones	• Si no se cumplen los criterios de validación para cada campo del formulario, el sistema mostrará un mensaje con el error correspondiente.
Comentarios	N.A.

Tabla A.17: Requisito 4.2 - Modificar datos de cita.

Requisito	Finalizar cita
Identificador	4.3
Prioridad	Alta
Descripción	Finaliza una cita existente en la aplicación.
Actor/es	Administrativo, cardiólogo
Entrada	• Identificador de la cita
Salida	• Cita existente marcada como finalizada. • Mensaje indicando el éxito de la operación.
Secuencia de Acciones	1. El actor solicita finalizar una cita existente; el sistema finaliza la cita y notifica de ello.
Precondición	El actor está autenticado en la aplicación.
Postcondición	La aplicación cuenta con una cita que ya existía, que ahora está marcada como finalizada.
Excepciones	N.A.
Comentarios	No elimina la cita, sino que se mantiene en la base de datos marcada como finalizada. Esto se puede dar tanto si es cancelada, como si ya ha sido realizada.

Tabla A.18: Requisito 4.3 - Finalizar cita.

Requisito	Ver cita
Identificador	4.4
Prioridad	Media
Descripción	Muestra la información de una cita.
Actor/es	Administrativo, cardiólogo
Entrada	Identificador de la cita
Salida	Todos los atributos de la cita.
Secuencia de Acciones	1. El actor solicita ver la vista con la información de la cita; el sistema muestra la vista con la información correspondiente.
Precondición	El actor está autenticado en la aplicación.
Postcondición	La información de la cita solicitada queda mostrada en pantalla.
Excepciones	N.A.
Comentarios	N.A.

Tabla A.19: Requisito 4.4 - Ver cita.

Requisito	Listar citas
Identificador	4.5
Prioridad	Media
Descripción	Lista todas las citas, con la posibilidad de aplicar filtros por atributos, y ordenar los resultados por atributos de forma ascendente o descendente.
Actor/es	Administrativo, cardiólogo
Entrada	• Atributos opcionales por los que filtrar y/o ordenar • Sentido de ordenación
Salida	• Lista de todas las citas tras aplicar filtros.
Secuencia de Acciones	1. El actor solicita la vista para listar citas; el sistema devuelve la vista correspondiente con la lista de citas. 2. El actor escoge los filtros y orden que quiere aplicar a los resultados, que pueden ser ninguno. 3. El actor solicita actualizar la lista de citas . 4. El sistema aplica los filtros y criterios de ordenación. 5. El sistema devuelve la lista de citas solicitada.
Precondición	El actor está autenticado en la aplicación.
Postcondición	La lista de citas con los filtros y criterios de ordenación solicitados queda mostrada en pantalla.
Excepciones	N.A.
Comentarios	N.A.

Tabla A.20: Requisito 4.5 - Listar citas.

A.E. Módulo de diagnósticos

Requisito	Crear diagnóstico
Identificador	5.1
Prioridad	Alta
Descripción	Crea un diagnóstico en la aplicación.
Actor/es	Administrativo, cardiólogo
Entrada	• Paciente • Nombre • Gravedad • Estado • Descripción • Fecha de diagnóstico • Cita asociada • Usuarios • Notas
Salida	• Nuevo diagnóstico con los datos de entrada en la aplicación. • Mensaje indicando el éxito de la operación.
Secuencia de Acciones	1. El actor solicita el formulario para crear un nuevo diagnóstico; el sistema muestra el formulario correspondiente. 2. El actor rellena los campos del formulario con la información correspondiente. 3. El actor solicita realizar la creación del diagnóstico. 4. El sistema valida los datos. 5. El sistema crea el nuevo diagnóstico y notifica de ello.
Precondición	El actor está autenticado en la aplicación.
Postcondición	La aplicación cuenta con un nuevo diagnóstico.
Excepciones	• Si no se cumplen los criterios de validación para cada campo del formulario, el sistema mostrará un mensaje con el error correspondiente.
Comentarios	El diagnóstico se crea a partir de un paciente; opcionalmente, en el formulario se puede asociar a una cita previa.

Tabla A.21: Requisito 5.1 - Crear diagnóstico.

Requisito	Modificar datos de diagnóstico
Identificador	5.2
Prioridad	Baja
Descripción	Modifica los datos de un diagnóstico existente en la aplicación.
Actor/es	Administrativo, cardiólogo
Entrada	• Identificador • Nombre • Gravedad • Descripción • Cita asociada • Usuarios • Detalles
Salida	• Diagnóstico existente modificado según los datos de entrada. • Mensaje indicando el éxito de la operación.
Secuencia de Acciones	1. El actor solicita el formulario para modificar un diagnóstico existente; el sistema muestra el formulario correspondiente. 2. El actor modifica los valores deseados. 3. El actor solicita realizar la modificación de los datos del diagnóstico. 4. El sistema valida los datos. 5. El sistema modifica los datos del diagnóstico y notifica de ello.
Precondición	El actor está autenticado en la aplicación.
Postcondición	La aplicación cuenta con un diagnóstico que ya existía, pero que posee uno o más atributos con valor distinto a los que tenía antes de la operación.
Excepciones	• Si no se cumplen los criterios de validación para cada campo del formulario, el sistema mostrará un mensaje con el error correspondiente .
Comentarios	N.A.

Tabla A.22: Requisito 5.2 - Modificar datos de diagnóstico.

Requisito	Ver diagnóstico
Identificador	5.3
Prioridad	Media
Descripción	Muestra la información de un diagnóstico.
Actor/es	Administrativo, cardiólogo
Entrada	Identificador del diagnóstico
Salida	Todos los atributos del diagnóstico.
Secuencia de Acciones	1. El actor solicita ver la vista con la información del diagnóstico; el sistema muestra la vista con la información correspondiente.
Precondición	El actor está autenticado en la aplicación.
Postcondición	La información del diagnóstico solicitado queda mostrada en pantalla.
Excepciones	N.A.
Comentarios	N.A.

Tabla A.23: Requisito 5.3 - Ver diagnóstico.

Requisito	Listar diagnósticos
Identificador	5.4
Prioridad	Media
Descripción	Lista todos los diagnósticos, con la posibilidad de aplicar filtros por atributos, y ordenar los resultados por atributos de forma ascendente o descendente.
Actor/es	Administrativo, cardiólogo
Entrada	• Atributos opcionales por los que filtrar y/o ordenar • Sentido de ordenación
Salida	• Lista de todos los diagnósticos tras aplicar filtros.
Secuencia de Acciones	1. El actor solicita la vista para listar diagnósticos; el sistema devuelve la vista correspondiente con la lista de diagnósticos. 2. El actor escoge los filtros y orden que quiere aplicar a los resultados, que pueden ser ninguno. 3. El actor solicita actualizar la lista de diagnósticos. 4. El sistema aplica los filtros y criterios de ordenación. 5. El sistema devuelve la lista de diagnósticos solicitada.
Precondición	El actor está autenticado en la aplicación.
Postcondición	La lista de diagnósticos con los filtros y criterios de ordenación solicitados queda mostrada en pantalla.
Excepciones	N.A.
Comentarios	N.A.

Tabla A.24: Requisito 5.4 - Listar diagnósticos.

A.F. Módulo de tratamientos

Requisito	Crear tratamiento
Identificador	6.1
Prioridad	Alta
Descripción	Crea un tratamiento en la aplicación.
Actor/es	Administrativo, cardiólogo
Entrada	• Paciente • Nombre • Estado • Descripción • Duración • Cita asociada • Diagnóstico asociado • Usuarios • Notas
Salida	• Nuevo tratamiento con los datos de entrada en la aplicación. • Mensaje indicando el éxito de la operación.
Secuencia de Acciones	1. El actor solicita el formulario para crear un nuevo tratamiento; el sistema muestra el formulario correspondiente. 2. El actor rellena los campos del formulario con la información correspondiente. 3. El actor solicita realizar la creación del tratamiento. 4. El sistema valida los datos. 5. El sistema crea el nuevo tratamiento y notifica de ello.
Precondición	El actor está autenticado en la aplicación.
Postcondición	La aplicación cuenta con un nuevo tratamiento
Excepciones	• Si no se cumplen los criterios de validación para cada campo del formulario, el sistema mostrará un mensaje con el error correspondiente.
Comentarios	El tratamiento se crea a partir de un paciente; opcionalmente, en el formulario se puede asociar a una cita previa.

Tabla A.25: Requisito 6.1 - Crear tratamiento.

Requisito	Modificar datos de tratamiento
Identificador	6.2
Prioridad	Baja
Descripción	Modifica los datos de un tratamiento existente en la aplicación.
Actor/es	Administrativo, cardiólogo
Entrada	• Identificador • Nombre • Descripción • Duración • Cita asociada • Usuarios • Notas
Salida	• Tratamiento existente modificado según los datos de entrada. • Mensaje indicando el éxito de la operación.
Secuencia de Acciones	1. El actor solicita el formulario para modificar un tratamiento existente; el sistema muestra el formulario correspondiente. 2. El actor modifica los valores deseados. 3. El actor solicita realizar la modificación de los datos del tratamiento. 4. El sistema valida los datos. 5. El sistema modifica los datos del tratamiento y notifica de ello.
Precondición	El actor está autenticado en la aplicación.
Postcondición	La aplicación cuenta con un tratamiento que ya existía, pero que posee uno o más atributos con valor distinto a los que tenía antes de la operación.
Excepciones	• Si no se cumplen los criterios de validación para cada campo del formulario, el sistema mostrará un mensaje con el error correspondiente.
Comentarios	N.A.

Tabla A.26: Requisito 6.2 - Modificar datos de tratamiento.

Requisito	Finalizar tratamiento
Identificador	6.3
Prioridad	Media
Descripción	Finaliza un tratamiento existente en la aplicación.
Actor/es	Administrativo, cardiólogo
Entrada	• Identificador del tratamiento
Salida	• Tratamiento existente marcado como finalizado. • Mensaje indicando el éxito de la operación.
Secuencia de Acciones	1. El actor solicita finalizar un tratamiento existente; el sistema finaliza el tratamiento y notifica de ello.
Precondición	El actor está autenticado en la aplicación.
Postcondición	La aplicación cuenta con un tratamiento que ya existía, que ahora está marcado como finalizado.
Excepciones	N.A.
Comentarios	No elimina el tratamiento, sino que se mantiene en la base de datos marcado como finalizado. Esto se puede dar tanto si es cancelado, como si ya ha sido terminado.

Tabla A.27: Requisito 6.3 - Finalizar tratamiento.

Requisito	Ver tratamiento
Identificador	6.4
Prioridad	Media
Descripción	Muestra la información de un tratamiento.
Actor/es	Administrativo, cardiólogo
Entrada	Identificador del tratamiento
Salida	Todos los atributos del tratamiento.
Secuencia de Acciones	1. El actor solicita ver la vista con la información del tratamiento; el sistema muestra la vista con la información correspondiente.
Precondición	El actor está autenticado en la aplicación.
Postcondición	La información del tratamiento solicitado queda mostrada en pantalla.
Excepciones	N.A.
Comentarios	N.A.

Tabla A.28: Requisito 6.4 - Ver tratamiento.

Requisito	Listar tratamientos
Identificador	6.5
Prioridad	Media
Descripción	Lista todos los tratamientos, con la posibilidad de aplicar filtros por atributos, y ordenar los resultados por atributos de forma ascendente o descendente.
Actor/es	Administrativo, cardiólogo
Entrada	• Atributos opcionales por los que filtrar y/o ordenar • Sentido de ordenación
Salida	• Lista de todos los tratamientos tras aplicar filtros.
Secuencia de Acciones	1. El actor solicita la vista para listar tratamientos; el sistema devuelve la vista correspondiente con la lista de tratamientos. 2. El actor escoge los filtros y orden que quiere aplicar a los resultados, que pueden ser ninguno. 3. El actor solicita actualizar la lista de tratamientos. 4. El sistema aplica los filtros y criterios de ordenación. 5. El sistema devuelve la lista de tratamientos solicitada.
Precondición	El actor está autenticado en la aplicación.
Postcondición	La lista de tratamientos con los filtros y criterios de ordenación solicitados queda mostrada en pantalla.
Excepciones	N.A.
Comentarios	N.A.

Tabla A.29: Requisito 6.5 - Listar tratamientos.

A.G. Módulo de documentos clínicos

Requisito	Crear documento clínico
Identificador	7.1
Prioridad	Alta
Descripción	Crea un documento clínico en la aplicación
Actor/es	Administrativo, cardiólogo
Entrada	• Título • Tipo • Archivos • Usuarios
Salida	• Nuevo documento clínico con los datos de entrada en la aplicación. • Mensaje indicando el éxito de la operación.
Secuencia de Acciones	1. El actor solicita el formulario para crear un nuevo documento clínico; el sistema muestra el formulario correspondiente. 2. El actor rellena los campos del formulario con la información correspondiente. 3. El actor solicita realizar la creación del documento clínico. 4. El sistema valida los datos. 5. El sistema crea el nuevo documento clínico y notifica de ello.
Precondición	El actor está autenticado en la aplicación.
Postcondición	La aplicación cuenta con un nuevo documento clínico.
Excepciones	• Si no se cumplen los criterios de validación para cada campo del formulario, el sistema mostrará un mensaje con el error correspondiente.
Comentarios	N.A.

Tabla A.30: Requisito 7.1 - Crear documento clínico.

...

Apéndice **B**

Título del apéndice B

Los apéndices son secciones al final del documento en las que se agrega texto con el objetivo de ampliar los contenidos del documento principal.

Este texto se puede encontrar en cáscaras/fin.typ. Si deseas eliminarlo, basta con comentar la línea correspondiente al final del archivo memoria.typ.

*–¿Qué te parece desto, Sancho? – Dijo Don Quijote –
Bien podrán los encantadores quitarme la ventura,
pero el esfuerzo y el ánimo, será imposible.*

*— Segunda parte del Ingenioso Caballero don Quijote de la Mancha
Miguel de Cervantes*

*–Buena está – dijo Sancho –; fírmela vuestra merced.
–No es menester firmarla – dijo Don Quijote–,
sino solamente poner mi rúbrica.*

*— Segunda parte del Ingenioso Caballero don Quijote de la Mancha
Miguel de Cervantes*